EDMLDS - C4 - Regularisation Assignment - Final Submission (HarGagan Sahai)

1. Data Understanding

1.1. Data Loading

```python
# Importing necessary libraries

import pandas as pd; import numpy as np; import matplotlib.pyplot as plt; import seaborn as sns

from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score, mean_absolute_error, mean_absolute_percentage_error
from sklearn.preprocessing import MultiLabelBinarizer
from sklearn.model_selection import train_test_split
from statsmodels.stats.outliers_influence import variance_inflation_factor
from sklearn.linear_model import LinearRegression, Lasso, Ridge
import warnings; warnings.filterwarnings('ignore')
```

Python

1.1.1. Load the dataset

1.1.1

Load the dataset

```python
# Load the data

df = pd.read_csv("Car_Price_Data.csv")
```

Python

```python
df.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 15915 entries, 0 to 15914
Data columns (total 23 columns):
 #   Column               Non-Null Count  Dtype
---  ------               --------------  -----
 0   make_model           15915 non-null  str
 1   body_type            15915 non-null  str
 2   price                15915 non-null  int64
 3   vat                  15915 non-null  str
 4   km                   15915 non-null  float64
 5   Type                 15915 non-null  str
 6   Fuel                 15915 non-null  str
 7   Gears                15915 non-null  float64
 8   Comfort_Convenience  15915 non-null  str
 9   Entertainment_Media  15915 non-null  str
 10  Extras               15915 non-null  str
 11  Safety_Security      15915 non-null  str
 12  age                  15915 non-null  float64
 13  Previous_Owners      15915 non-null  float64
 14  hp_kW                15915 non-null  float64
 15  Inspection_new       15915 non-null  int64
 16  Paint_Type           15915 non-null  str
 17  Upholstery_type      15915 non-null  str
 18  Gearing_Type         15915 non-null  str
 19  Displacement_cc      15915 non-null  float64
...
 21  Drive_chain          15915 non-null  str
 22  cons_comb            15915 non-null  float64
dtypes: float64(8), int64(2), str(13)
memory usage: 2.8 MB
```

Output is truncated. View as a scrollable element or open in a text editor. Adjust cell output settings...

Changed the columns to lowercase for ease of access later.

```python
# While some of the column names are small case, others are capital case. Lets make all small case.
df.columns = [col.lower() for col in df.columns]
```

```python
df.head()
```

Python

	make_model	body_type	price	vat	km	type	fuel	gears	comfort_convenience	entertainment_media	...	previous_owners	hp_kw	inspection_new	paint_type	upholstery_type	gearing_type	displacement_cc	weight_kg	dr
0	Audi A1	Sedans	15770	VAT deductible	56013.0	Used	Diesel	7.0	Air conditioning,Armrest,Automatic climate con...	Bluetooth,Hands-free equipment,On-board comput...	...	2.0	66.0	1	Metallic	Cloth	Automatic	1422.0	1220.0	
1	Audi A1	Sedans	14500	Price negotiable	80000.0	Used	Benzine	7.0	Air conditioning,Automatic climate control,Hil...	Bluetooth,Hands-free equipment,On-board comput...	...	1.0	141.0	0	Metallic	Cloth	Automatic	1798.0	1255.0	
2	Audi A1	Sedans	14640	VAT deductible	83450.0	Used	Diesel	7.0	Air conditioning,Cruise control,Electrical sid...	MP3,On-board computer	...	1.0	85.0	0	Metallic	Cloth	Automatic	1598.0	1135.0	
3	Audi A1	Sedans	14500	VAT deductible	73000.0	Used	Diesel	6.0	Air suspension,Armrest,Auxiliary heating,Elect...	Bluetooth,CD player,Hands-free equipment,MP3,O...	...	1.0	66.0	0	Metallic	Cloth	Automatic	1422.0	1195.0	
4	Audi A1	Sedans	16790	VAT deductible	16200.0	Used	Diesel	7.0	Air conditioning,Armrest,Automatic climate con...	Bluetooth,CD player,Hands-free equipment,MP3,O...	...	1.0	66.0	1	Metallic	Cloth	Automatic	1422.0	1135.0	

5 rows × 23 columns

2. Analysis and Feature Engineering

2.1. Preliminary Analysis and Frequency Distributions

2.1.1. Check and fix missing values

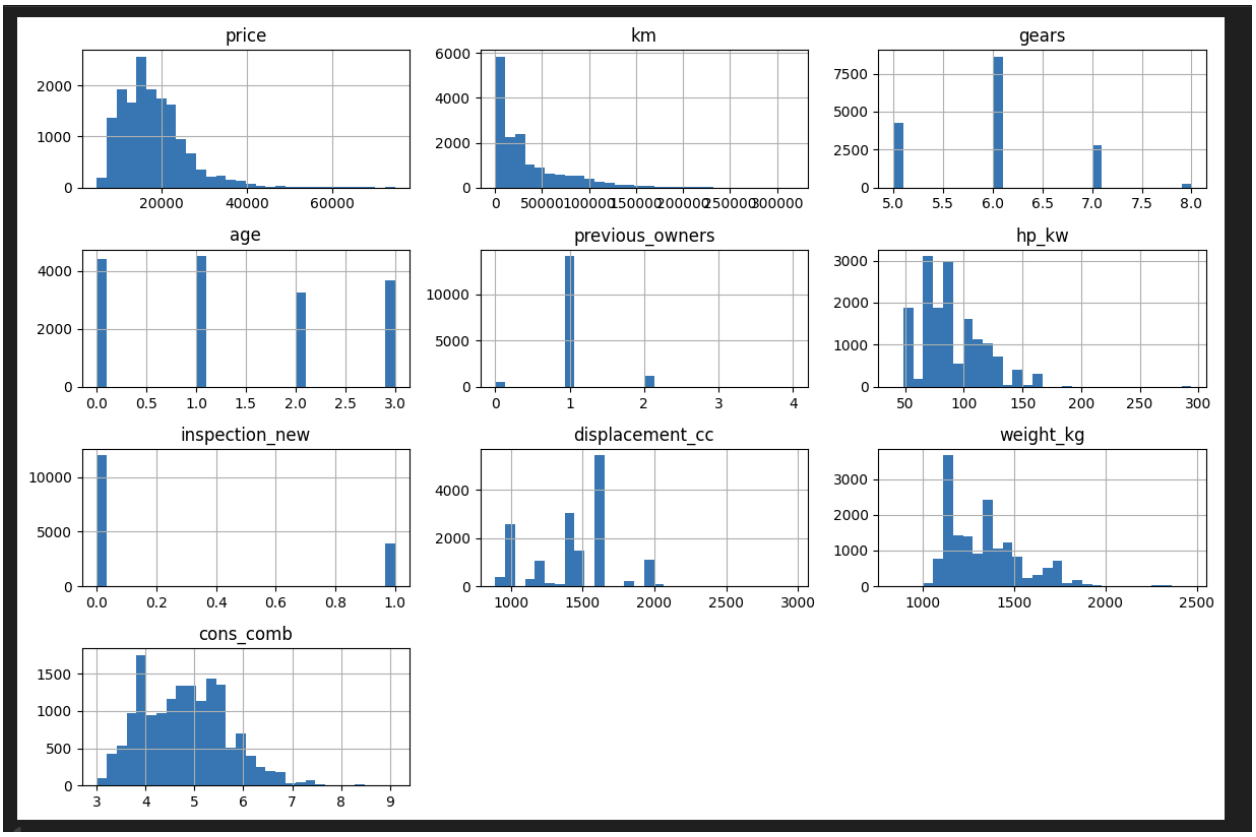
```
Missing value proportion (%) by column:

missing_pct

No missing values found.
```

No missing value found in the dataset.

2.1.2. Identify numerical predictors and plot their frequency distributions

Some observations from the distribution plot:

- Price distribustion is skewed with most cars are priced less than 40000
- Most of the cars are having single owners
- Higher numbers of cars have 1500cc
- Cars in the dataset are in the range of 0 to 3 years



2.1.3. Identify categorical predictors and plot their frequency distributions

The categorical columns are identified here. These are :

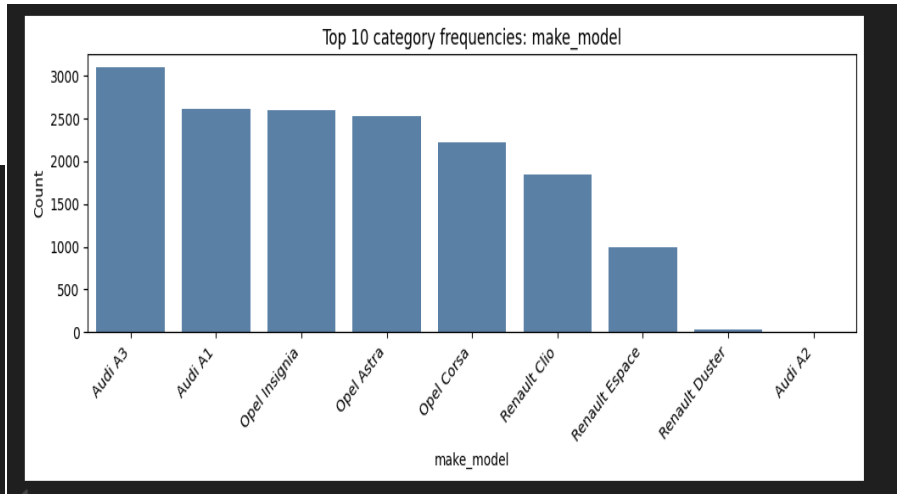
'make_model', 'body_type', 'vat', 'type', 'fuel', 'comfort_convenience', 'entertainment_media', 'extras', 'safety_security', 'paint_type', 'upholstery_type', 'gearing_type', 'drive_chain'

```
Python

Categorical columns:
['make_model', 'body_type', 'vat', 'type', 'fuel', 'comfort_convenience', 'entertainment_media', 'extras', 'safety_security', 'paint_type', 'upholstery_type', 'gearing_type', 'drive_chain']

Top categories for make_model:
```

Individual frequency distribution of these columns –

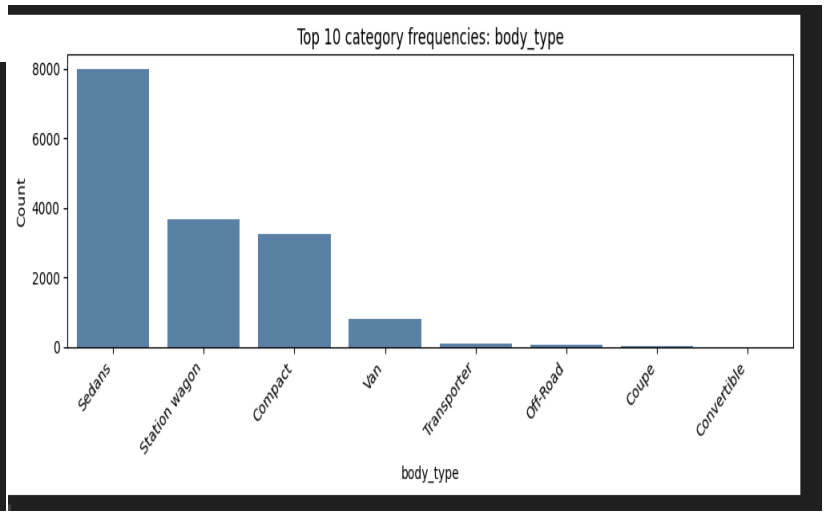
For ‘make model’ -

```
make_model
Audi A3            3097
Audi A1            2614
Opel Insignia      2598
Opel Astra         2525
Opel Corsa         2216
Renault Clio       1839
Renault Espace      991
Renault Duster       34
Audi A2               1
Name: count, dtype: int64
```



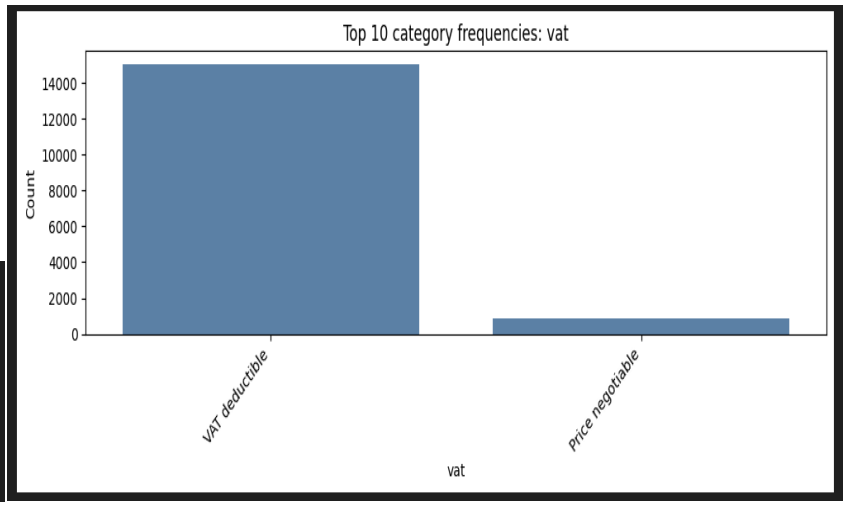
For body_type:

```
Top categories for body_type:

body_type
Sedans          8004
Station wagon   3677
Compact         3240
Van              817
Transporter       88
Off-Road          56
Coupe             25
Convertible        8
Name: count, dtype: int64
```



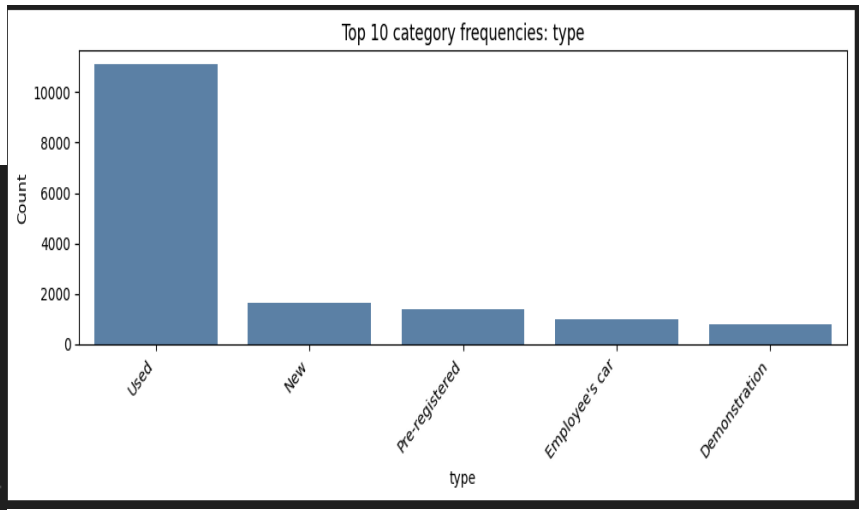
For Vat:

```
Top categories for vat:

vat
VAT deductible      15044
Price negotiable      871
Name: count, dtype: int64
```



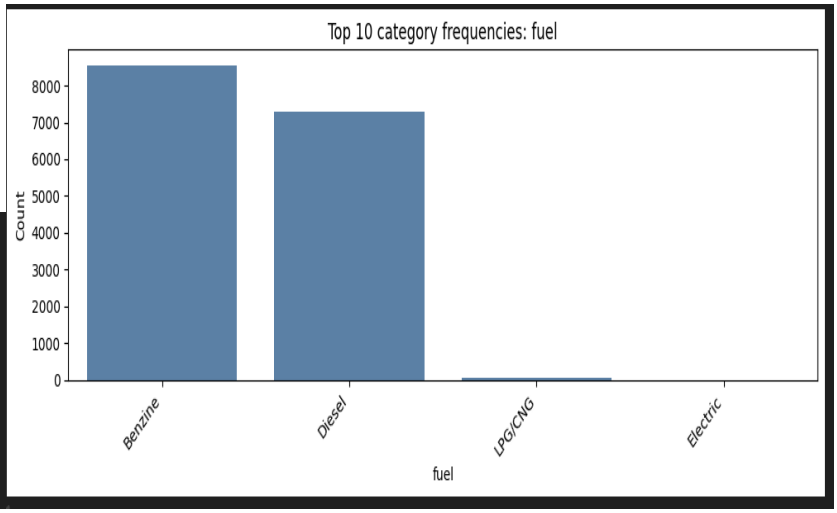
For Type:

```
Top categories for type:

type
Used                11095
New                  1649
Pre-registered       1364
Employee's car       1011
Demonstration         796
Name: count, dtype: int64
```



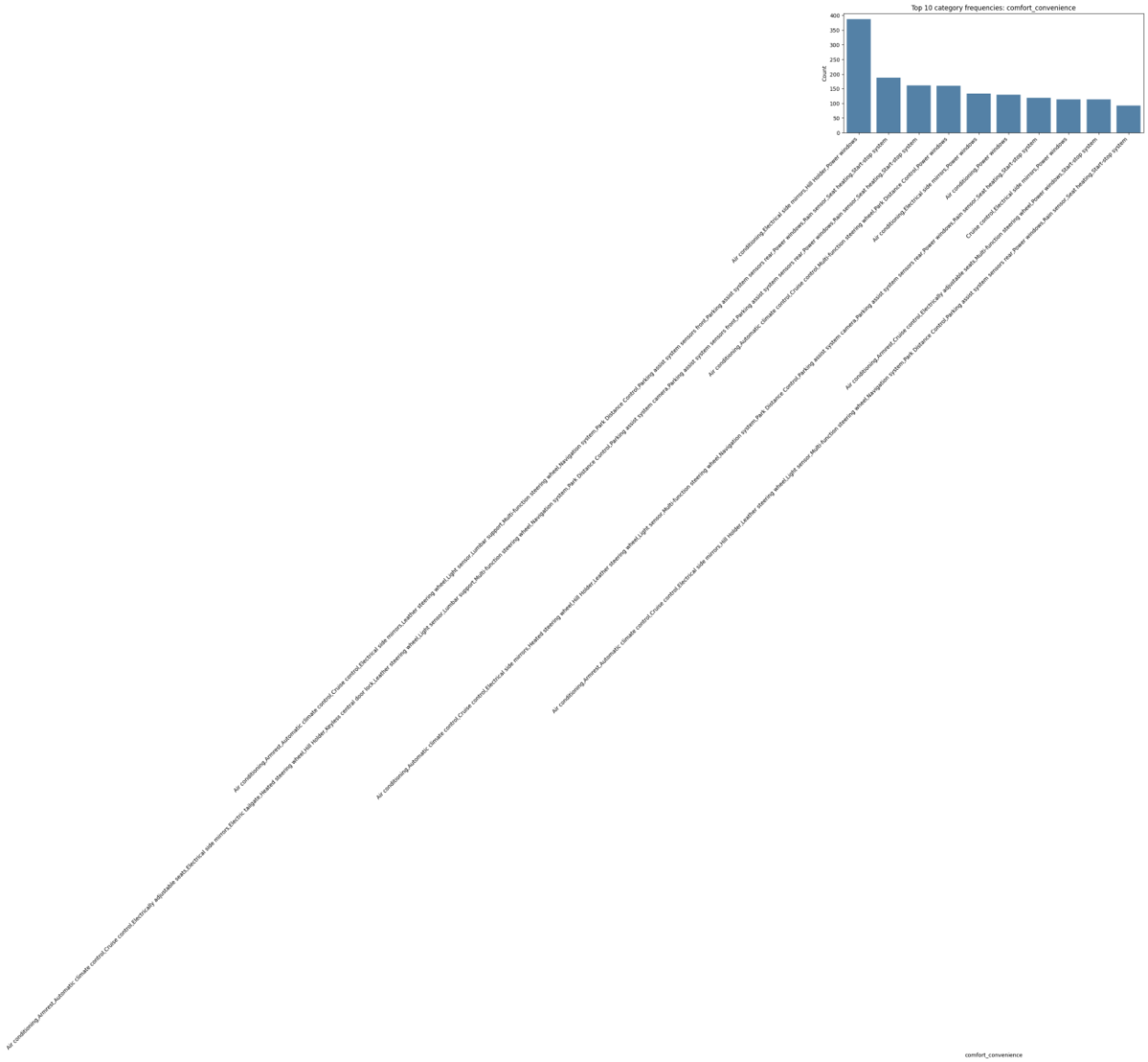
For Fuel:

```
Top categories for fuel:

fuel
Benzine     8548
Diesel      7298
LPG/CNG       64
Electric       5
Name: count, dtype: int64
```



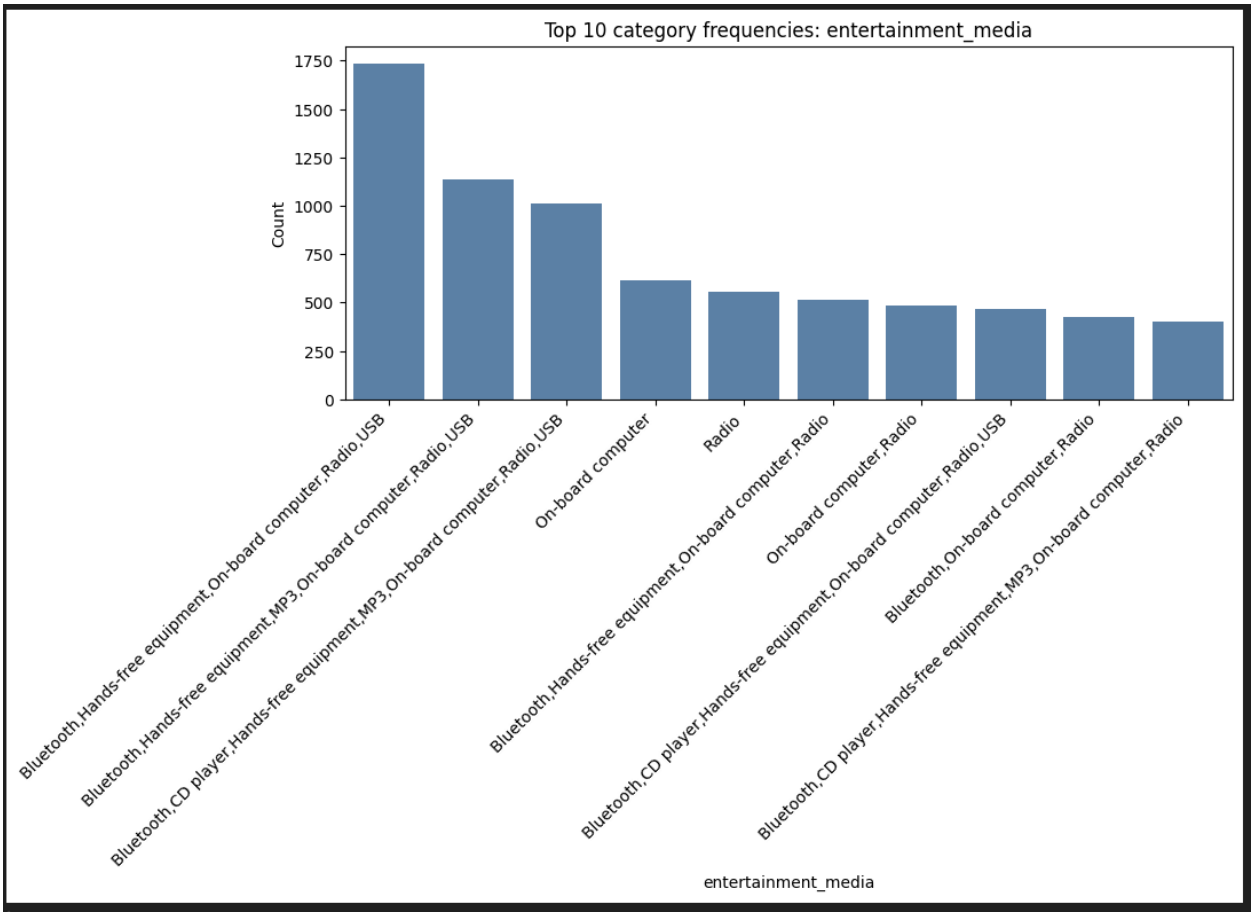
For comfort_convenience:

```
Top categories for comfort_convenience:

comfort_convenience
Air conditioning,Electrical side mirrors,Hill Holder,Power windows
Air conditioning,Armrest,Automatic climate control,Cruise control,Electrical side mirrors,Leather steering wheel,Light sensor,Lumbar support,Multi-function steering wheel,Navigation system,Park Distance Control,Parking assist system s
Air conditioning,Armrest,Automatic climate control,Cruise control,Electrically adjustable seats,Electrical side mirrors,Electric tailgate,Heated steering wheel,Hill Holder,Keyless central door lock,Leather steering wheel,Light sensor,
Air conditioning,Automatic climate control,Cruise control,Multi-function steering wheel,Park Distance Control,Power windows
Air conditioning,Electrical side mirrors,Power windows
Air conditioning,Power windows
Air conditioning,Automatic climate control,Cruise control,Electrical side mirrors,Heated steering wheel,Hill Holder,Leather steering wheel,Light sensor,Multi-function steering wheel,Navigation system,Park Distance Control,Parking assi
Cruise control,Electrical side mirrors,Power windows
Air conditioning,Armrest,Cruise control,Electrically adjustable seats,Multi-function steering wheel,Power windows,Start-stop system
Air conditioning,Armrest,Automatic climate control,Cruise control,Electrical side mirrors,Hill Holder,Leather steering wheel,Light sensor,Multi-function steering wheel,Navigation system,Park Distance Control,Parking assist system sens
Name: count, dtype: int64
```



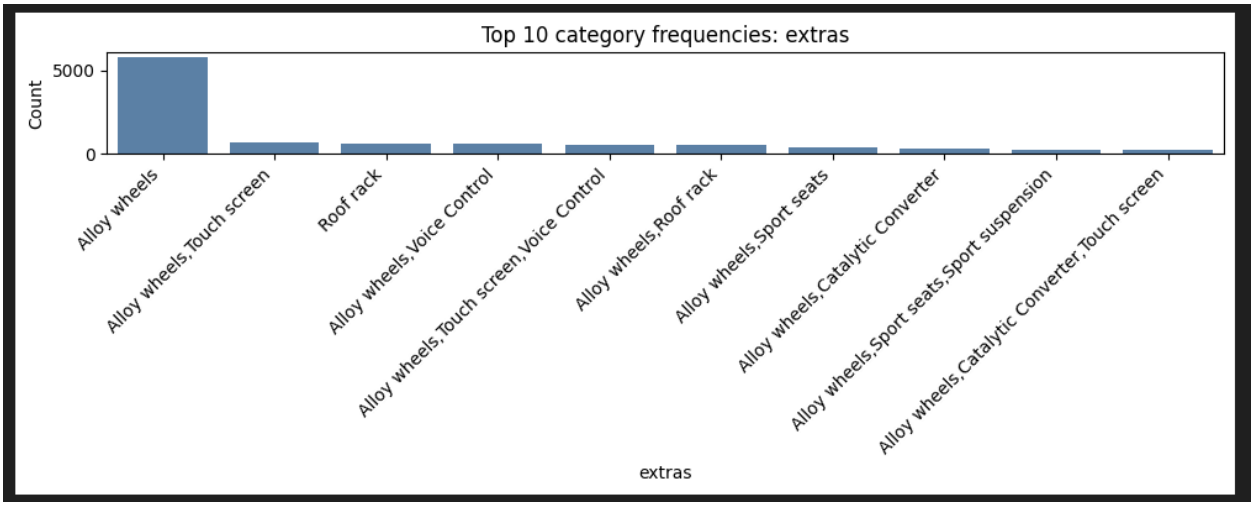
For entertainment_media:

```
Top categories for entertainment_media:

entertainment_media
Bluetooth,Hands-free equipment,On-board computer,Radio,USB                  1737
Bluetooth,Hands-free equipment,MP3,On-board computer,Radio,USB              1134
Bluetooth,CD player,Hands-free equipment,MP3,On-board computer,Radio,USB    1009
On-board computer                                                            615
Radio                                                                        558
Bluetooth,Hands-free equipment,On-board computer,Radio                       515
On-board computer,Radio                                                      487
Bluetooth,CD player,Hands-free equipment,On-board computer,Radio,USB         466
Bluetooth,On-board computer,Radio                                            426
Bluetooth,CD player,Hands-free equipment,MP3,On-board computer,Radio         404
Name: count, dtype: int64
```



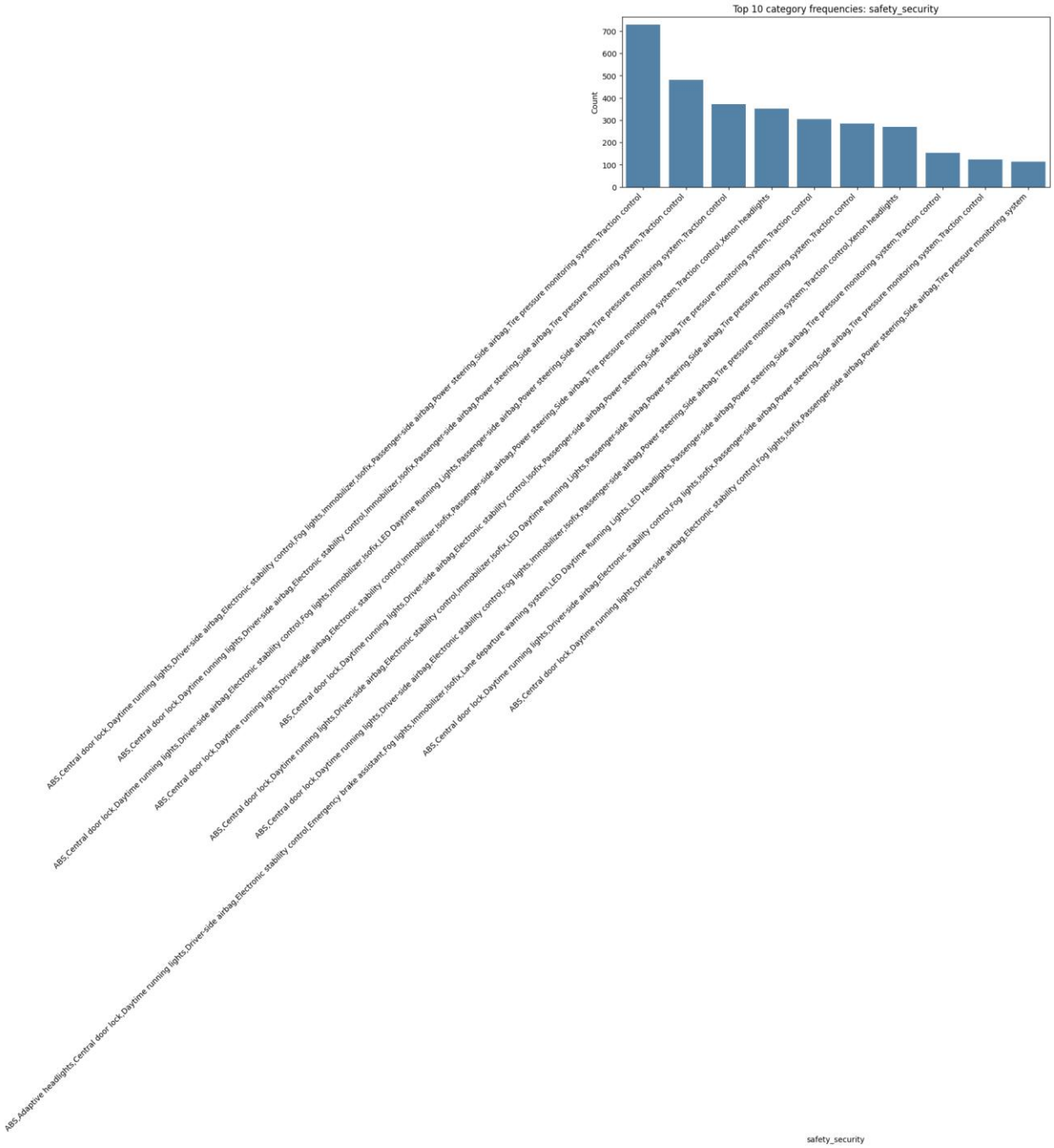
For extras:

```
extras
Alloy wheels                                        5785
Alloy wheels,Touch screen                            697
Roof rack                                            595
Alloy wheels,Voice Control                           582
Alloy wheels,Touch screen,Voice Control              544
Alloy wheels,Roof rack                               529
Alloy wheels,Sport seats                             382
Alloy wheels,Catalytic Converter                     275
Alloy wheels,Sport seats,Sport suspension            234
Alloy wheels,Catalytic Converter,Touch screen        228
Name: count, dtype: int64
```



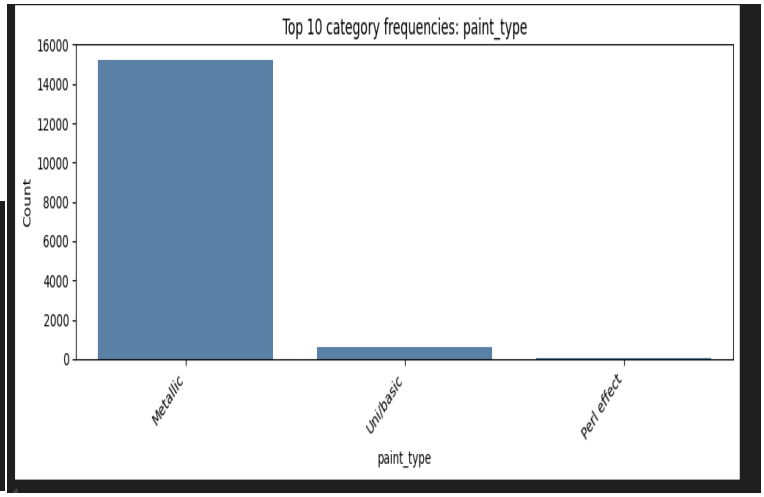
For safety_security:

```
Top categories for safety_security:

safety_security
ABS,Central door lock,Daytime running lights,Driver-side airbag,Electronic stability control,Fog lights,Immobilizer,Isofix,Passenger-side airbag,Power steering,Side airbag,Tire pressure monitoring system,Traction control
ABS,Central door lock,Daytime running lights,Driver-side airbag,Electronic stability control,Immobilizer,Isofix,Passenger-side airbag,Power steering,Side airbag,Tire pressure monitoring system,Traction control
ABS,Central door lock,Daytime running lights,Driver-side airbag,Electronic stability control,Fog lights,Immobilizer,Isofix,LED Daytime Running Lights,Passenger-side airbag,Power steering,Side airbag,Tire pressure monitoring system,Tra
ABS,Central door lock,Daytime running lights,Driver-side airbag,Electronic stability control,Immobilizer,Isofix,Passenger-side airbag,Power steering,Side airbag,Tire pressure monitoring system,Traction control,Xenon headlights
ABS,Central door lock,Daytime running lights,Driver-side airbag,Electronic stability control,Isofix,Passenger-side airbag,Power steering,Side airbag,Tire pressure monitoring system,Traction control
ABS,Central door lock,Daytime running lights,Driver-side airbag,Electronic stability control,Immobilizer,Isofix,LED Daytime Running Lights,Passenger-side airbag,Power steering,Side airbag,Tire pressure monitoring system,Traction contr
ABS,Central door lock,Daytime running lights,Driver-side airbag,Electronic stability control,Fog lights,Immobilizer,Isofix,Passenger-side airbag,Power steering,Side airbag,Tire pressure monitoring system,Traction control,Xenon headlig
ABS,Adaptive headlights,Central door lock,Daytime running lights,Driver-side airbag,Electronic stability control,Emergency brake assistant,Fog lights,Immobilizer,Isofix,Lane departure warning system,LED Daytime Running Lights,LED Head
ABS,Central door lock,Daytime running lights,Driver-side airbag,Electronic stability control,Fog lights,Isofix,Passenger-side airbag,Power steering,Side airbag,Tire pressure monitoring system,Traction control
ABS,Central door lock,Daytime running lights,Driver-side airbag,Electronic stability control,Fog lights,Isofix,Passenger-side airbag,Power steering,Side airbag,Tire pressure monitoring system
Name: count, dtype: int64
```



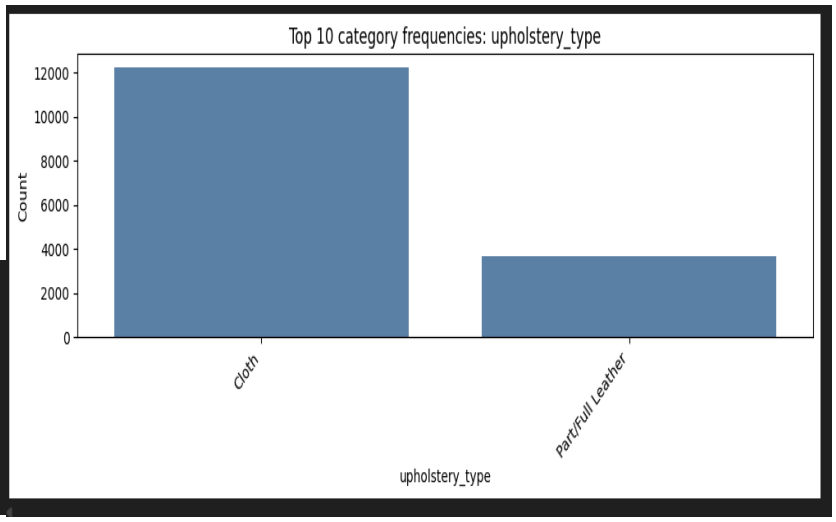
For paint_type:

```
Top categories for paint_type:

paint_type
Metallic       15246
Uni/basic        637
Perl effect       32
Name: count, dtype: int64
```



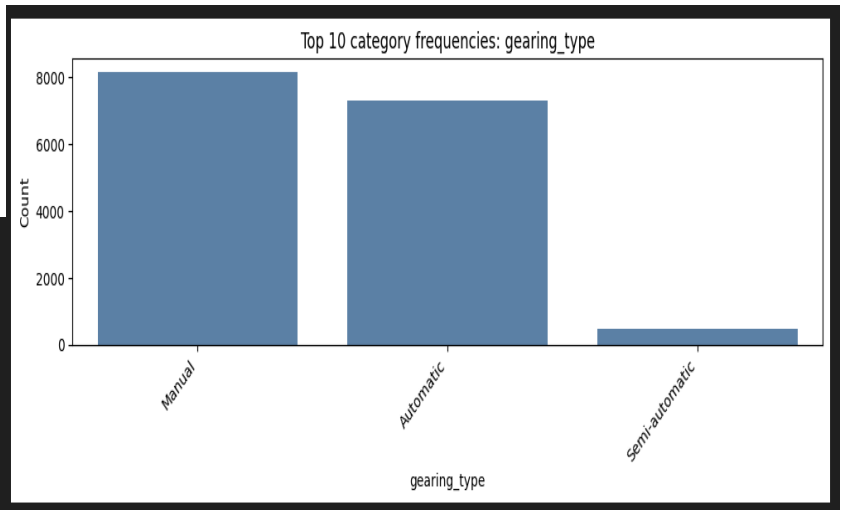
For upholstery_type

```
Top categories for upholstery_type:

upholstery_type
Cloth                12234
Part/Full Leather     3681
Name: count, dtype: int64
```



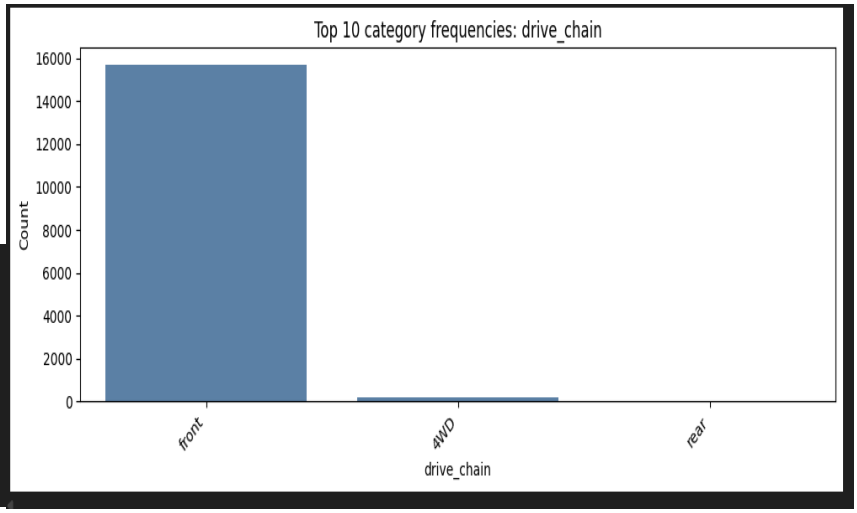
For gearing_type:

```
Top categories for gearing_type:

gearing_type
Manual            8149
Automatic         7297
Semi-automatic     469
Name: count, dtype: int64
```



For drive_chain:

```
Top categories for drive_chain:

drive_chain
front    15707
4WD        204
rear         4
Name: count, dtype: int64
```



For features **comfort_convenience, entertainment_media, extras, and safety_security**, analyzed their individual entries and got frequency of each feature inside the multi-valued column entries:

Top 10 individual features count for **comfort_convenience:**

- Air conditioning 15095
- Power windows 14756
- Electrical side mirrors 12488
- Multi-function steering wheel 11744
- Cruise control 11622
- Park Distance Control 10655
- Parking assist system sensors rear 10086
- Leather steering wheel 9735
- Start-stop system 9455

- Automatic climate control 9197

Top 10 individual features count for **entertainment_media:**

- On-board computer 13222
- Radio 13196
- Bluetooth 12685
- Hands-free equipment 10371
- USB 9019
- MP3 6258
- CD player 5024
- Sound system 2244
- Digital radio 2241
- Television 39

Top 10 individual features count for **'extras'**:

- Alloy wheels 13994
- Voice Control 4338
- Touch screen 4050
- Sport seats 3104
- Roof rack 3053
- Catalytic Converter 2262
- Sport suspension 1622
- Sport package 1201
- Trailer hitch 658
- Shift paddles 508

Top 10 individual features count for **safety_security**:

- ABS 15022
- Driver-side airbag 14801
- Power steering 14489
- Passenger-side airbag 14333
- Electronic stability control 14077
- Side airbag 13998
- Central door lock 13575
- Isofix 12689
- Traction control 11845
- Tire pressure monitoring system 11577

Summary of feature-spec columns:

	column	unique_combinations	avg_features_per_car	max_features_in_row	unique_individual_features
0	comfort_convenience	6196	12.19	33	38
1	entertainment_media	346	4.67	10	10
2	extras	659	2.24	10	17
3	safety_security	4442	12.79	29	29

2.1.4. Fix columns with low frequency values and class imbalances

```python
# Combine similar categories
df['type'] = df['type'].replace({
    'Pre-registered': 'Nearly New',
    "Employee's car": 'Nearly New',
    'Demonstration': 'Nearly New',
    'New': 'Nearly New'
})

# Check class distribution
print(df['type'].value_counts())
```

```
type
Used          11095
Nearly New     4820
Name: count, dtype: int64
```

Assigned the pre-registered, emplyee's car, demo car and new are all changed as 'nearly new' to reduce car imbalance.

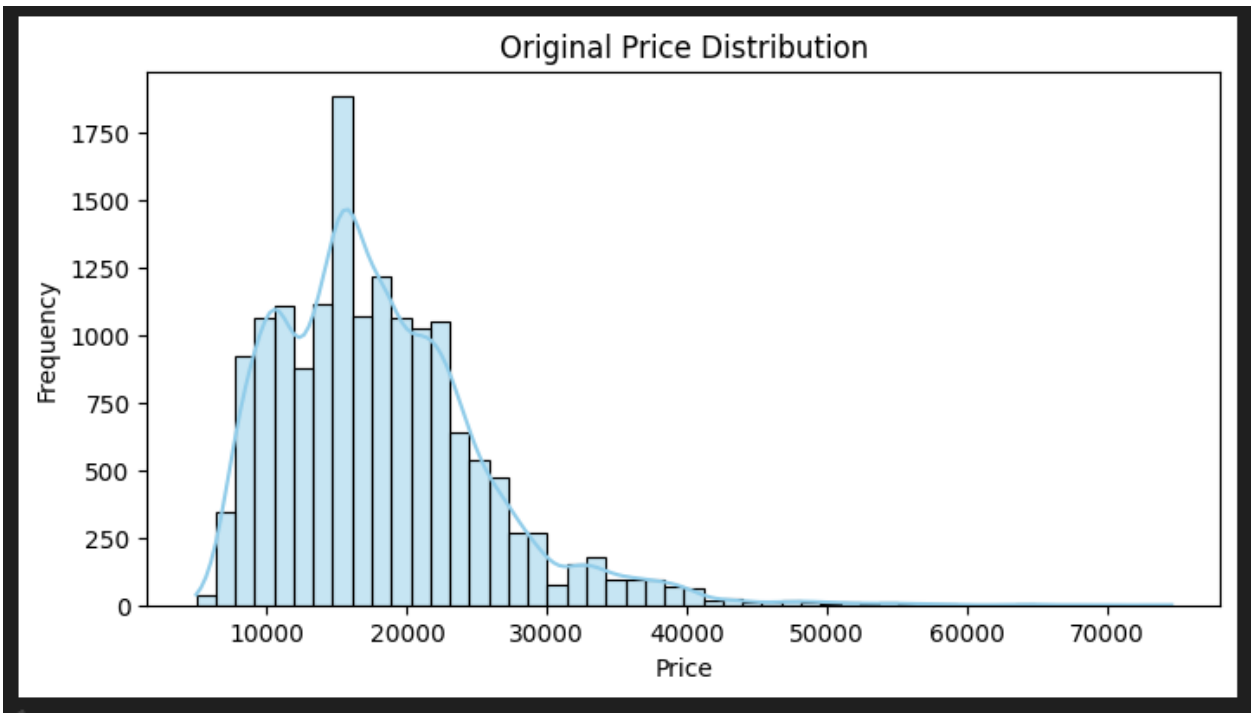
Approach for fixing other low frequency values and class imbalance

Identified 3 other columns with similar class imbalance and low frequency issue as 'type'. Those are 'body_type', 'paint_type' and 'fuel' column.

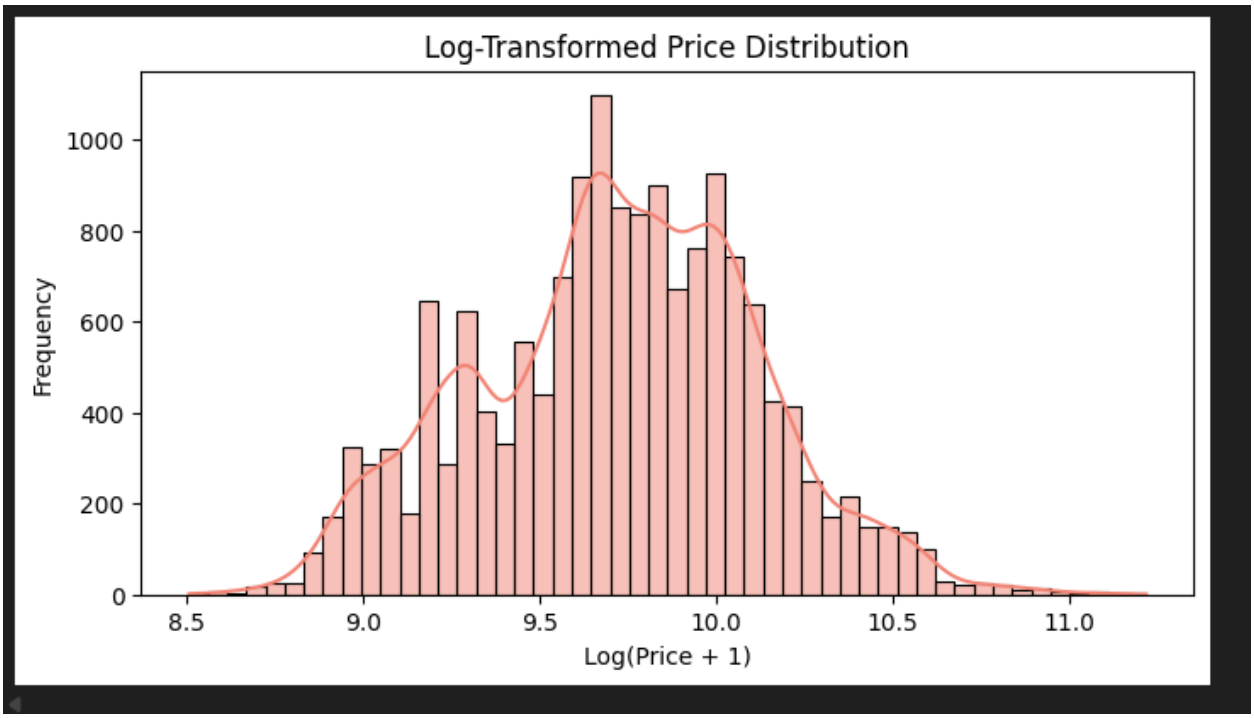
Using a minimum value count for each of those columns

```
body_type
Sedans           8004
Station wagon    3677
Compact          3240
Other             994
Name: count, dtype: int64
fuel
Benzine    8548
Diesel     7298
Other        69
Name: count, dtype: int64
paint_type
Metallic    15246
Other         669
Name: count, dtype: int64
```

2.1.5. Identify target variable and plot the frequency distributions. Apply necessary transformations.



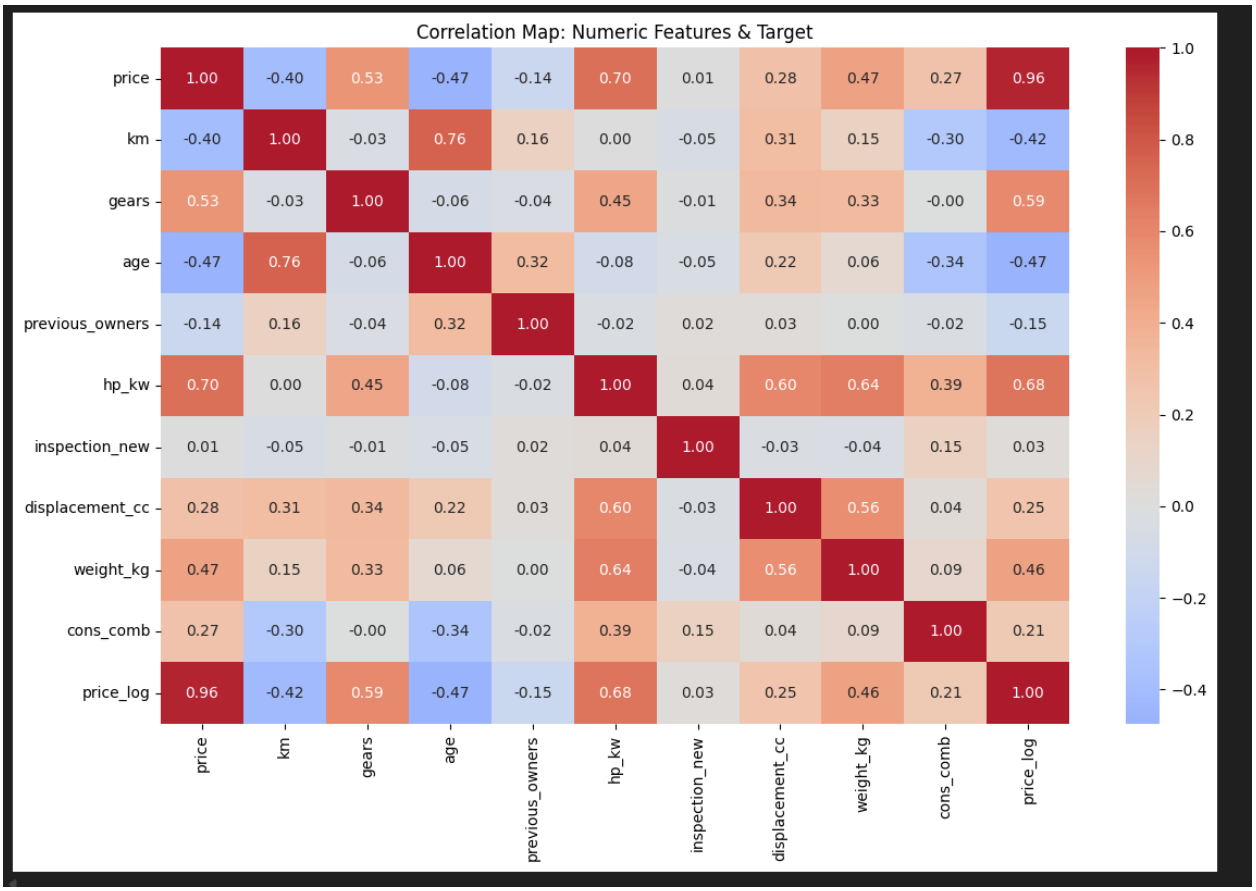
Above graph shows price is pretty skewed, with most of them falling in the range of 10k to 40k.



Log transformation applied to reduce the skewedness and it look much more normalized now. Using that feature (price_log) in further analysis as target feature.

2.2. Correlation analysis

2.2.1. Plot the correlation map between features and target variable



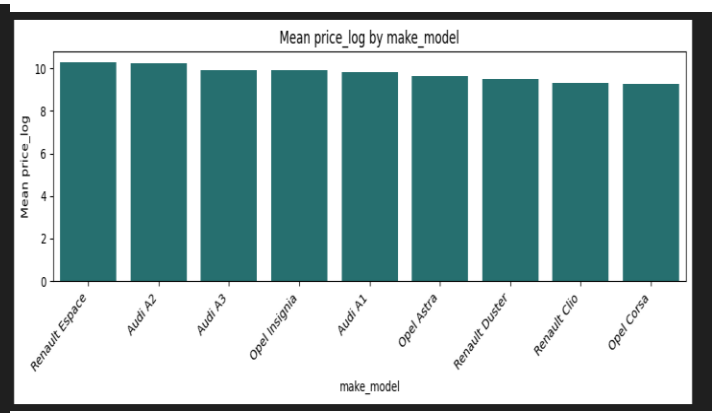

```
Correlation with target variable:

price_log          1.000000
price              0.962281
hp_kw              0.678024
gears              0.588194
weight_kg          0.464597
displacement_cc    0.254854
cons_comb          0.211097
inspection_new     0.030927
previous_owners   -0.152029
km                -0.419189
age               -0.474618
Name: price_log, dtype: float64
```

2.2.2. Analyse correlation between categorical features and target variable

Considering the target variable as price_log, correlation analysis between other categorical features with that is given below:

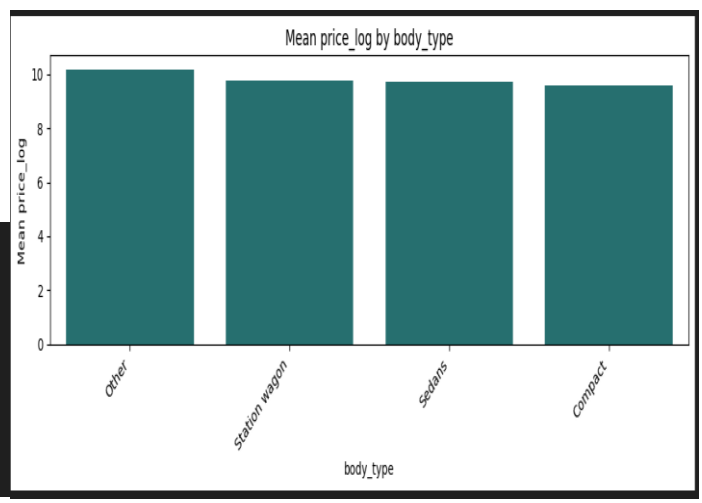
For price_log vs make_model:

```
Mean price_log for each category in make_model:

make_model
Renault Espace    10.271724
Audi A2           10.247113
Audi A3            9.928165
Opel Insignia      9.912658
Audi A1            9.817643
Opel Astra         9.626041
Renault Duster     9.483063
Renault Clio       9.335822
Opel Corsa         9.276059
Name: price_log, dtype: float64
```

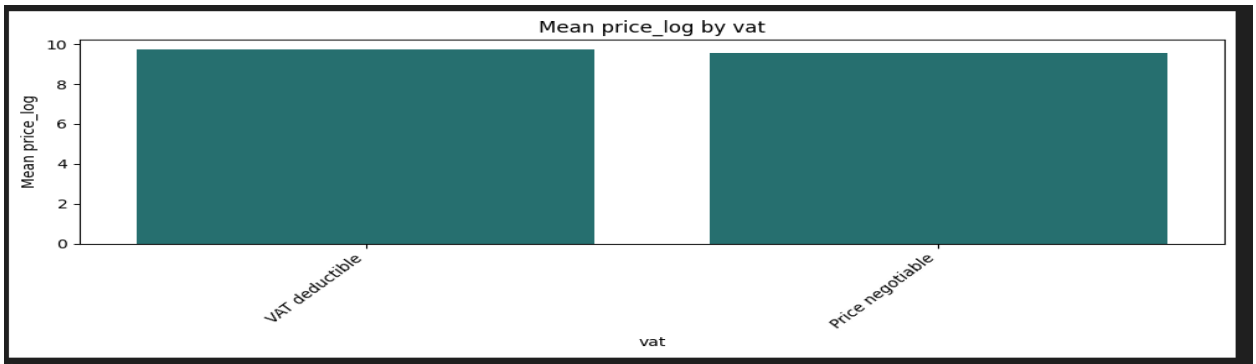


For price_log vs body_type:

```
Mean price_log for each category in body_type:

body_type
Other           10.174948
Station wagon    9.753456
Sedans           9.716144
Compact          9.555735
Name: price_log, dtype: float64
```

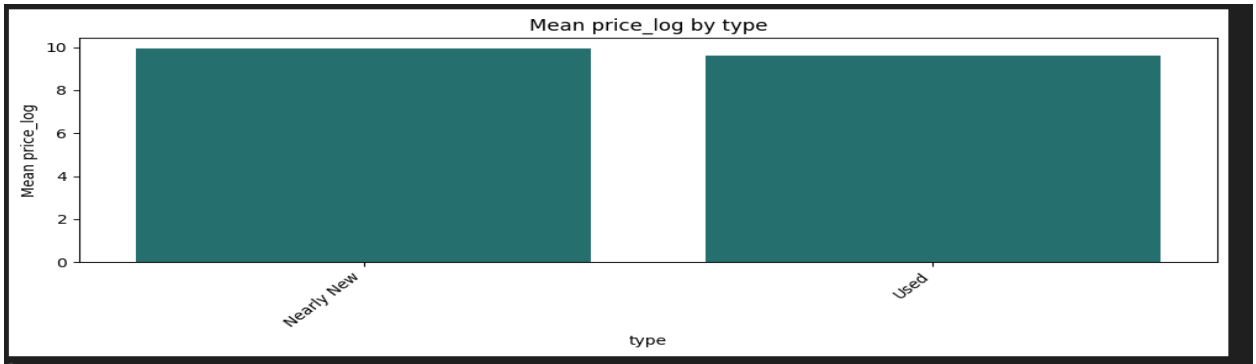


For price_log vs vat:



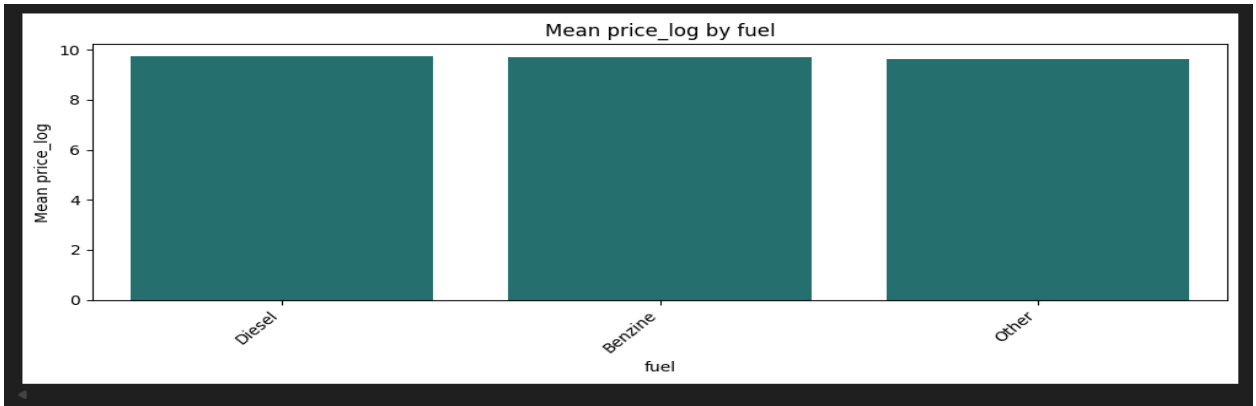

```
Mean price_log for each category in vat:

vat
VAT deductible      9.730286
Price negotiable    9.556299
Name: price_log, dtype: float64
```

For price_log vs type (condition):



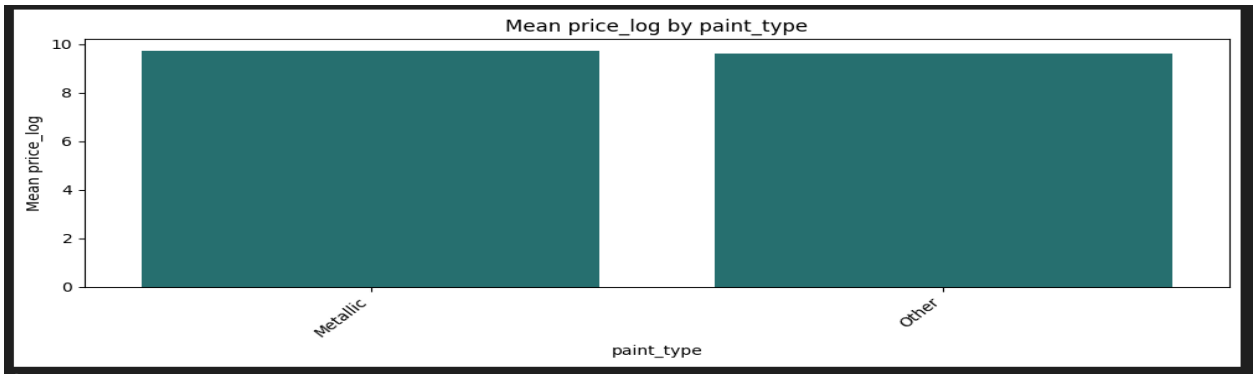

```
Mean price_log for each category in type:

type
Nearly New    9.928387
Used          9.630566
Name: price_log, dtype: float64
```

For price_log vs fuel:



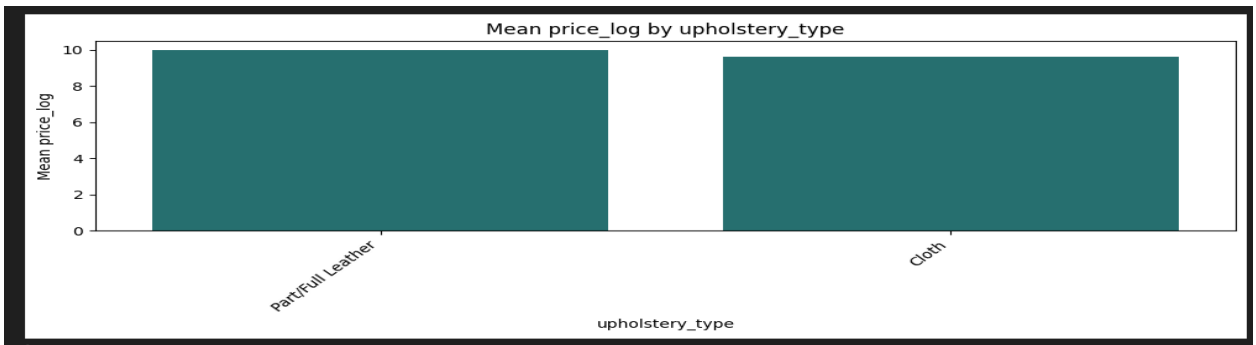

```
Mean price_log for each category in fuel:

fuel
Diesel     9.734926
Benzine    9.709328
Other      9.639463
Name: price_log, dtype: float64
```

For price_log vs paint_type:



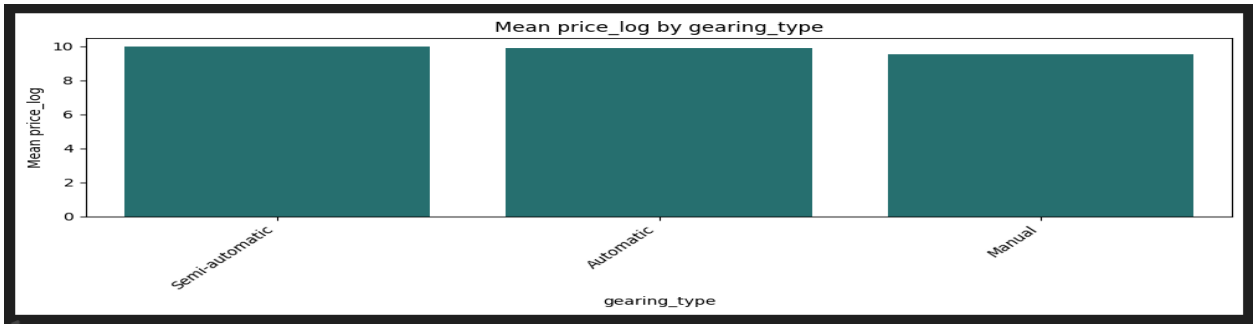

```
Mean price_log for each category in paint_type:

paint_type
Metallic    9.725404
Other       9.615009
Name: price_log, dtype: float64
```

For price_log vs upholstery_type:



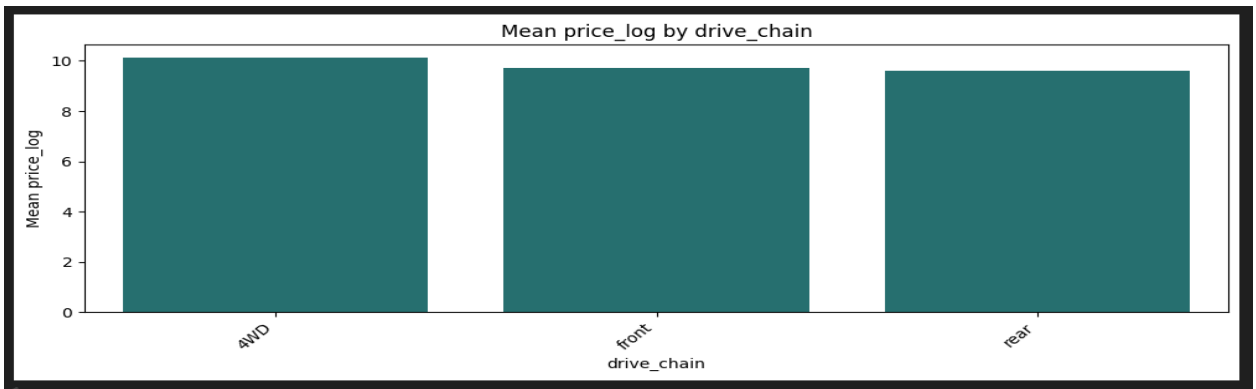

```
Mean price_log for each category in upholstery_type:

upholstery_type
Part/Full Leather    9.981383
Cloth                9.642348
Name: price_log, dtype: float64
```

For price_log vs gearing_type:




```
Mean price_log for each category in gearing_type:

gearing_type
Semi-automatic    9.972004
Automatic         9.906177
Manual            9.540276
Name: price_log, dtype: float64
```

For drive_chain:

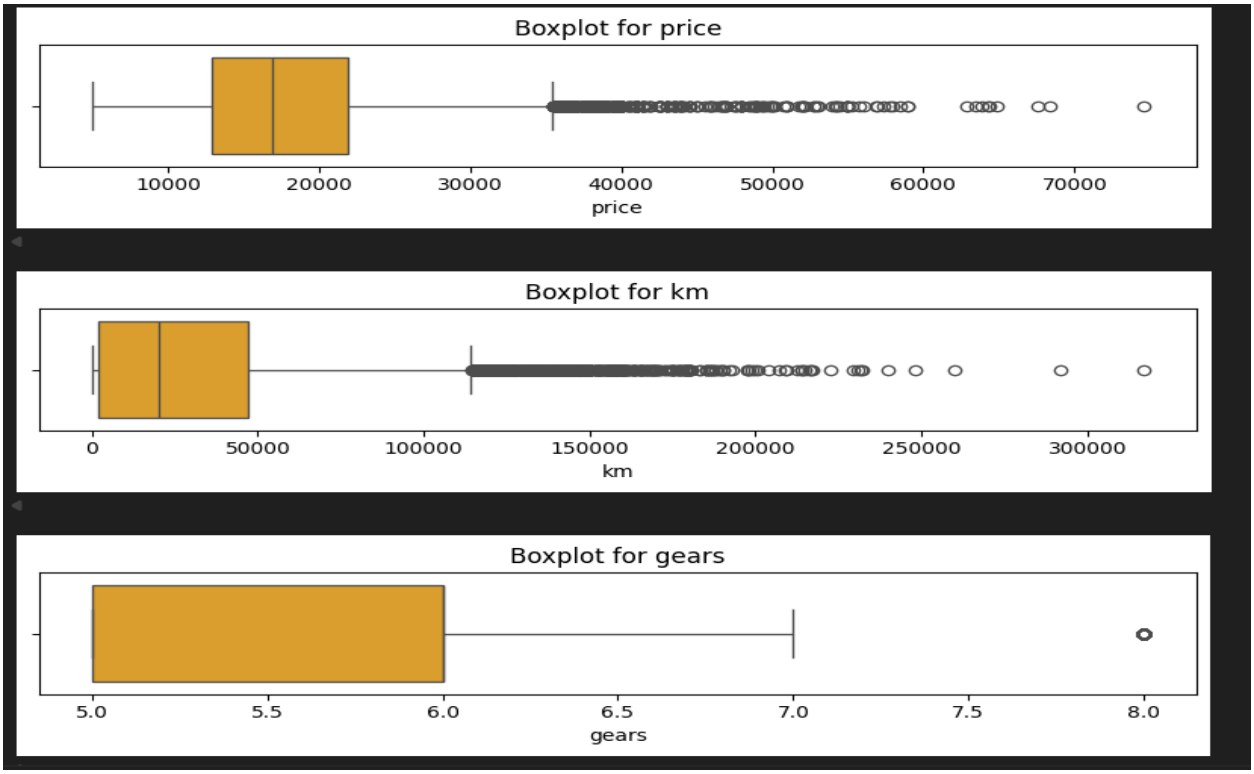


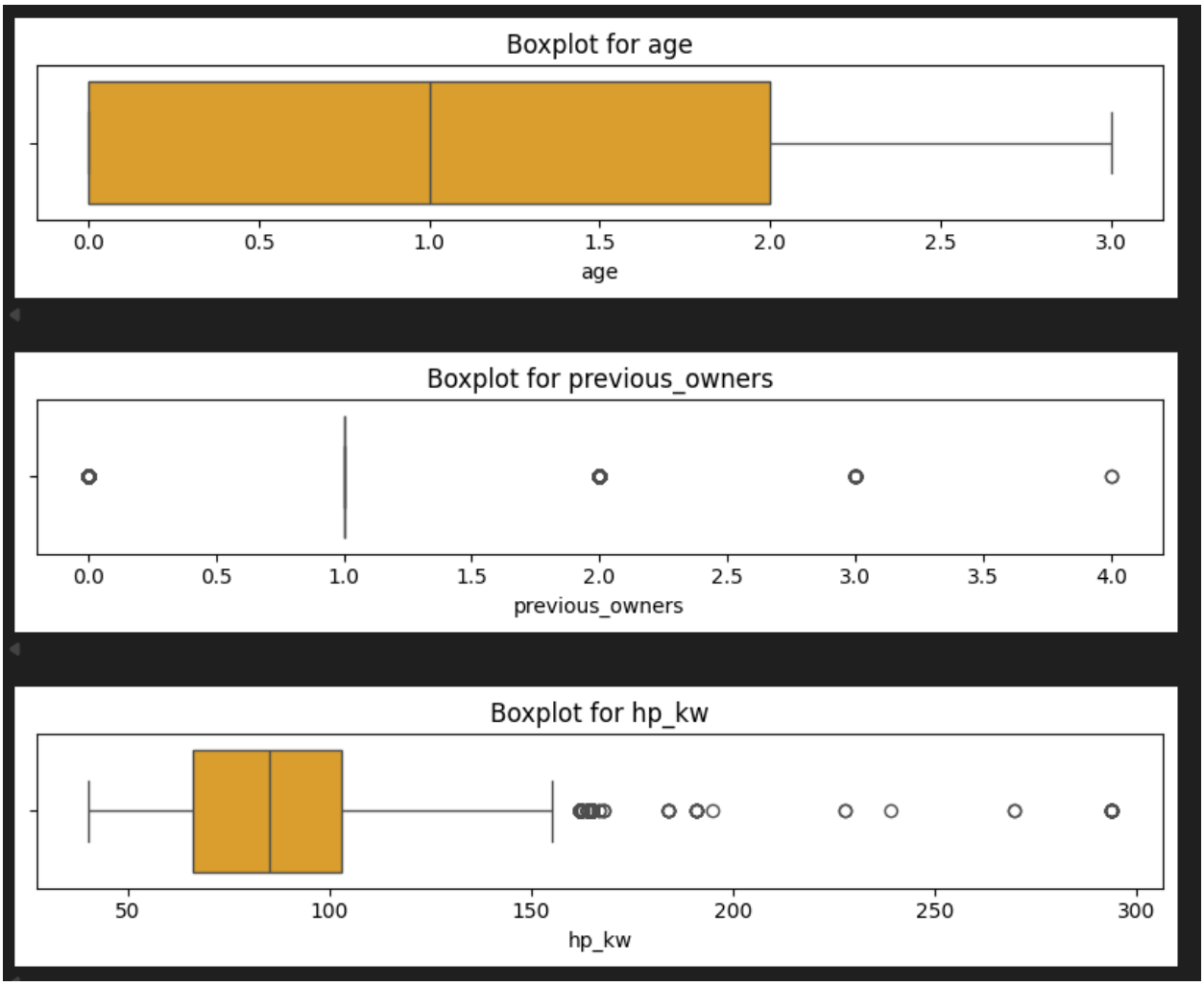

```
Mean price_log for each category in drive_chain:

drive_chain
4WD      10.133491
front     9.715432
rear      9.606166
Name: price_log, dtype: float64
```

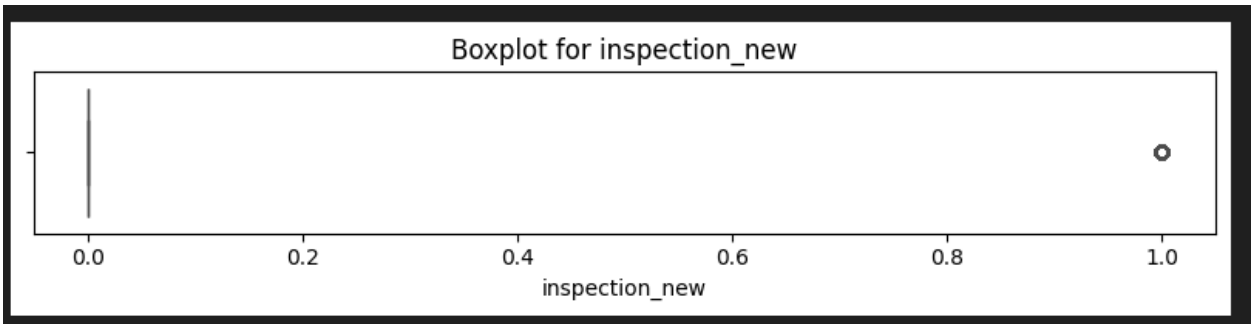
2.3. Outlier analysis

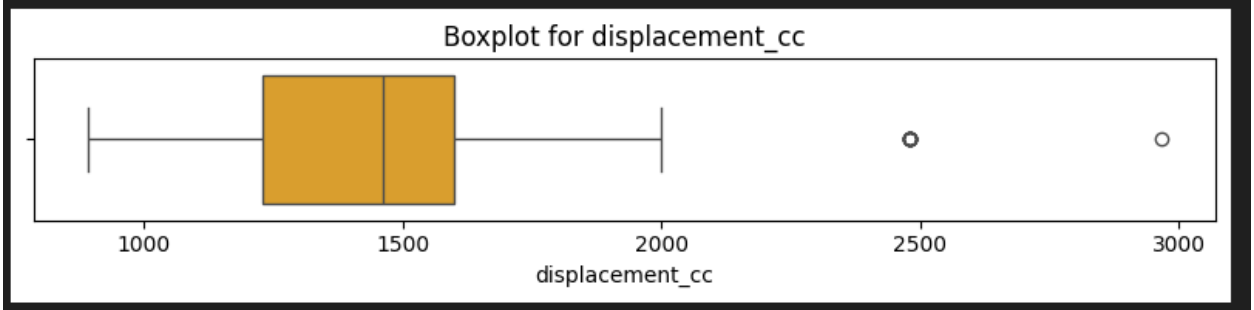
2.3.1. Identify potential outliers in the data

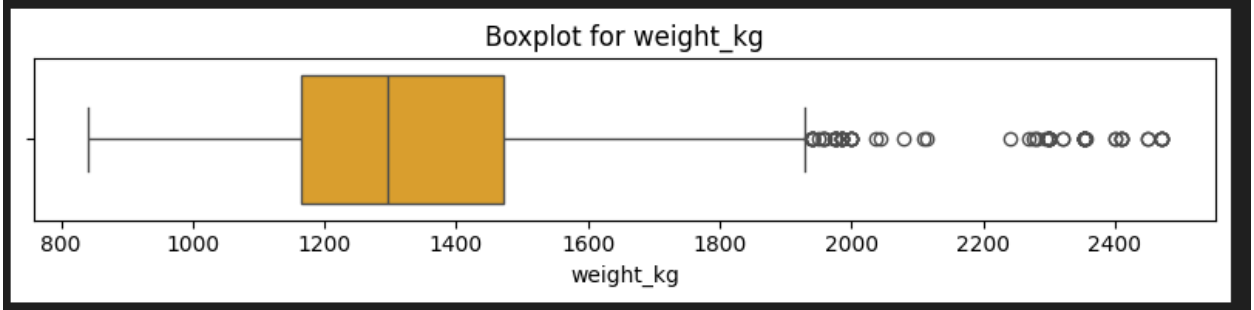
Below box-plots shows the outlier analysis for different numerical features in the dataset. This analysis is used later to get rid of the outliers.

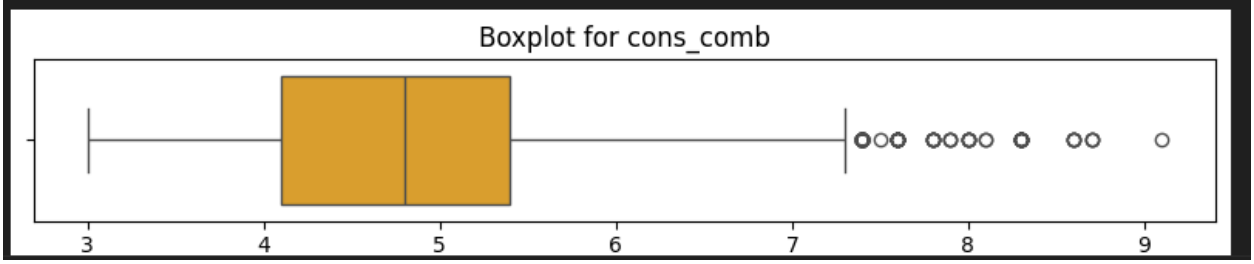


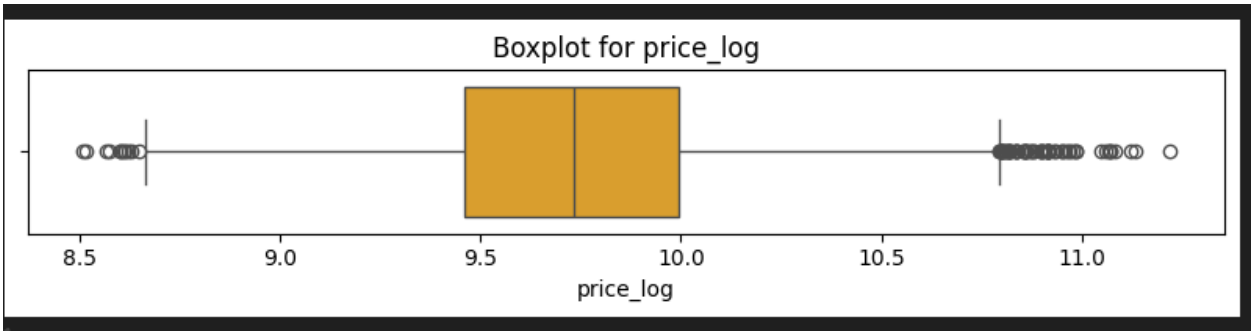

Boxplot for age
0.0
0.5
1.0
1.5
2.0
2.5
3.0
age
Boxplot for previous_owners
0.0
0.5
1.0
1.5
2.0
2.5
3.0
3.5
4.0
previous_owners
Boxplot for hp_kw
50
100
150
200
250
300
hp_kw


Boxplot for inspection_new
0.0
0.2
0.4
0.6
0.8
1.0
inspection_new


Boxplot for displacement_cc
1000
1500
2000
2500
3000
displacement_cc


Boxplot for weight_kg
800
1000
1200
1400
1600
1800
2000
2200
2400
weight_kg


Boxplot for cons_comb
3
4
5
6
7
8
9


Boxplot for price_log
8.5
9.0
9.5
10.0
10.5
11.0
price_log

```
Outlier counts for each numeric column:
price: 479 outliers
km: 689 outliers
gears: 225 outliers
age: 0 outliers
previous_owners: 1757 outliers
hp_kw: 361 outliers
inspection_new: 3932 outliers
displacement_cc: 21 outliers
weight_kg: 87 outliers
cons_comb: 125 outliers
price_log: 71 outliers
```

2.3.2. Handle the outliers suitably

```
Data shape after outlier removal: (9317, 24)
```

Outliers removed by getting the lower and upper bound by using the IQR handling approach. By getting the inter-quartile range, and then getting the higher and lower bounds to remove the outliers from the data.

2.4. Feature Engineering

2.4.1. Fix any redundant columns and create new ones if needed

```
Column 'previous_owners' is redundant (only one unique value).
Column 'inspection_new' is redundant (only one unique value).
Dropped redundant columns: ['previous_owners', 'inspection_new']
Created 'age_category' column.
Created 'power_weight_ratio' column.
```

2.4.2. Analysis and feature engineering on ['Comfort_Convenience', 'Entertainment_Media', 'Extras', 'Safety_Security']

```
Features in 'comfort_convenience' present in >80% of cars: 2
['Air conditioning', 'Power windows']

Features in 'comfort_convenience' present in <5% of cars: 11
['Electrically heated windshield', 'Parking assist system self-steering', 'Panorama roof', 'Heads-up display', 'Auxiliary heating', 'Sunroof', 'Massage seats', 'Air suspension', 'Wind deflector', 'Leather seats', 'Windshield']

Features in 'entertainment_media' present in >80% of cars: 1
['On-board computer']

Features in 'entertainment_media' present in <5% of cars: 1
['Television']

Features in 'extras' present in >80% of cars: 1
['Alloy wheels']

Features in 'extras' present in <5% of cars: 9
['Trailer hitch', 'Shift paddles', 'Ski bag', 'Winter tyres', 'Cab or rented Car', 'Handicapped enabled', 'Tuned car', 'Right hand drive', 'Sliding door']

Features in 'safety_security' present in >80% of cars: 7
['ABS', 'Driver-side airbag', 'Power steering', 'Passenger-side airbag', 'Side airbag', 'Electronic stability control', 'Central door lock']

Features in 'safety_security' present in <5% of cars: 2
['Driver drowsiness detection', 'Night view assist']
```

Recommendation

We are taking 80% of the features and would be dropping the ones which are present in less than 5% of cars.

Out of these features, we will check the ones which are present in most of the cars or are absent from most of the cars. These kinds of features can be removed as they just increase the dimensionality without explaining the variance.

```python
# Drop features from df
for col in spec_cols:
    rare_feats = rare_features_dict[col]
    def filter_rare_features(row):
        features = [f.strip() for f in str(row).split(',') if f.strip()]
        filtered = [f for f in features if f not in rare_feats]
        return ', '.join(filtered)
    df[col] = df[col].apply(filter_rare_features)
```

2.4.3. Perform feature encoding

```
Applied get_dummies to columns: ['make_model', 'body_type', 'vat', 'type', 'fuel', 'paint_type', 'upholstery_type', 'gearing_type', 'drive_chain', 'age_category']
```

2.4.4. Split the data into training and testing sets

```
Train shape: (7453, 102), Test shape: (1864, 102)
```

2.4.5. Scale the features

```
Scaled train shape: (7453, 102), Scaled test shape: (1864, 102)
```

Using MinMaxScaler() scaled the data.

3. Linear Regression Models

3.1. Baseline Linear Regression Model

3.1.1. Build and fit a basic linear regression model. Perform evaluation using suitable metrics

	Actual Train	Predicted Train
1359	9.923339	9.88
3498	10.125871	10.13
998	9.735128	9.68
4609	10.165506	10.18
4600	10.199919	10.19

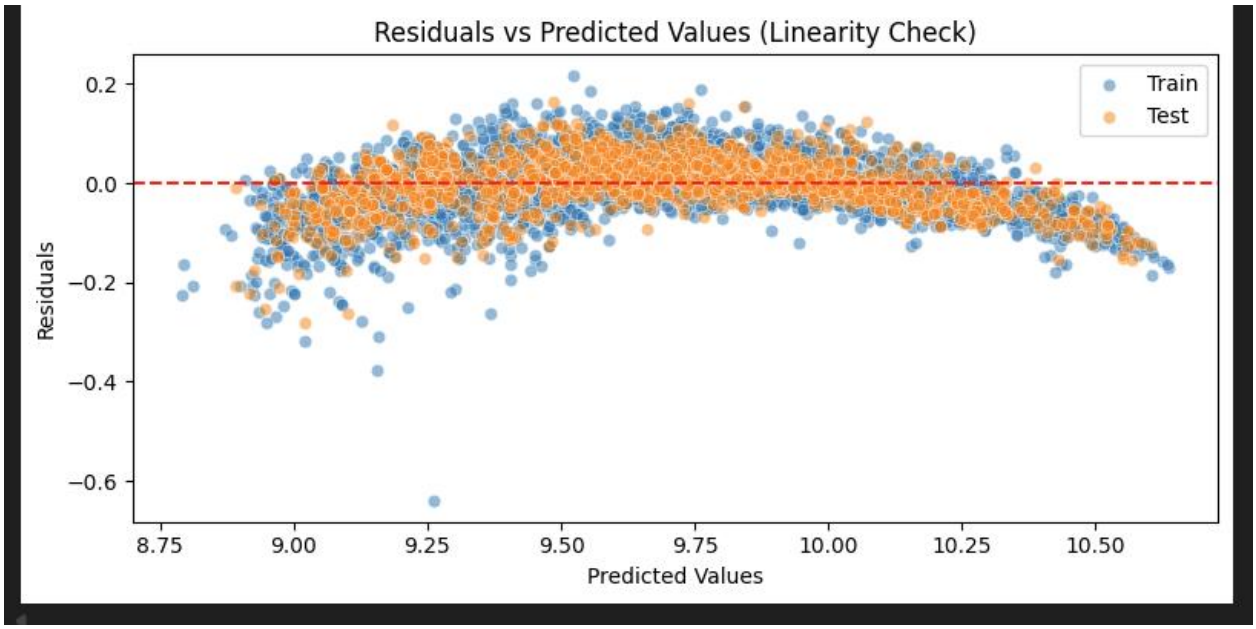
	Actual Test	Predicted Test
8358	9.200391	9.17
6209	9.332646	9.24
4793	9.711176	9.67
2191	9.994288	10.04
1044	9.729194	9.73

Dataset	MSE	Var	RMSE	Std Dev	MAE	Mean	MAPE	R^2
Training	0.003	0.141	0.051	0.376	0.038	9.690	0.394	0.982
Testing	0.003	0.142	0.050	0.377	0.038	9.688	0.396	0.982

From the above we can observe the following:

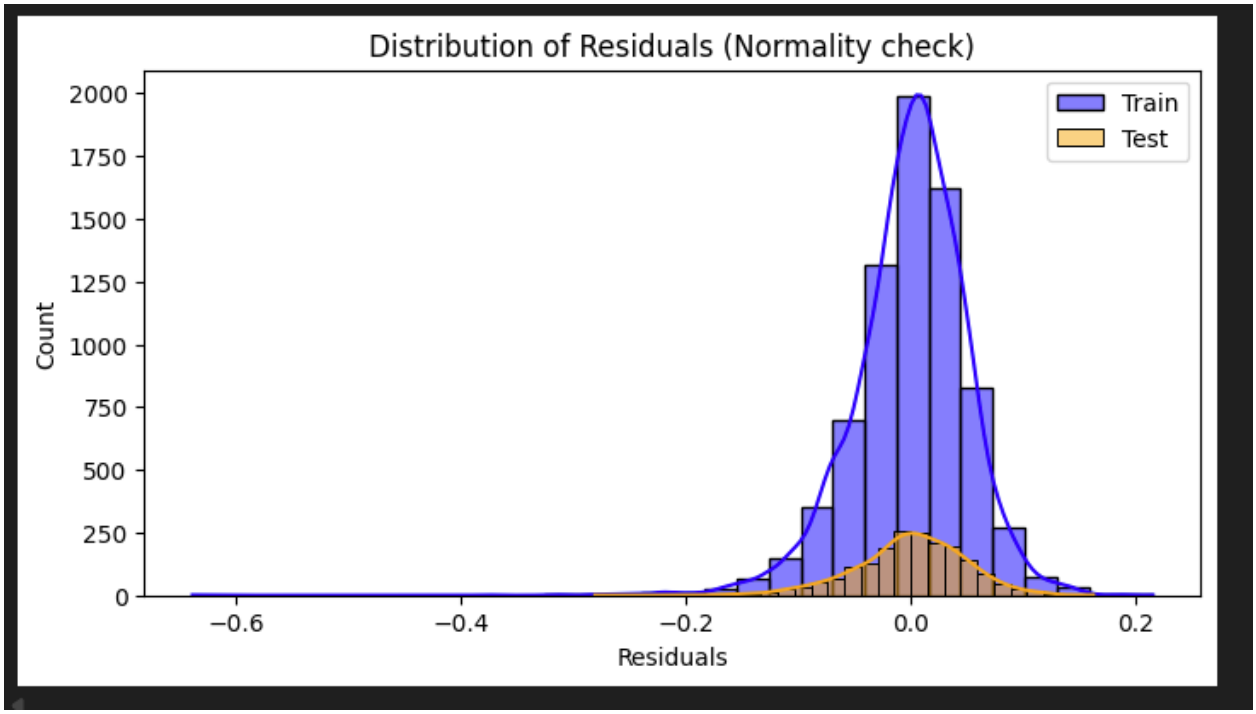
- The model achieves excellent performance, with low error metrics (MSE, RMSE, MAE, MAPE).
- R2 of 0.98 suggests the model captures almost all relevant patterns in the data.
- Similar scores for training and testing indicate minimal overfitting.
- Features and preprocessing steps are effective for price prediction.

3.1.2. Analyze residuals and check other assumptions of linear regression



Few observations from the plot:

- The model is systematically underpredicting in the middle and overpredicting at the extremes.
- The model shows non-linearity— the model is missing some curvature in the relationship between features and the target.
- Both Train (blue) and Test (orange) follow the same pattern, and it suggests the issue has **model bias**, not overfitting
- The vertical spread of residuals is:
    - Wider around the middle (≈ 9.3–9.8)
    - Narrower toward the extremes (low and high predictions)
    - Shows **mild heteroscedasticity**

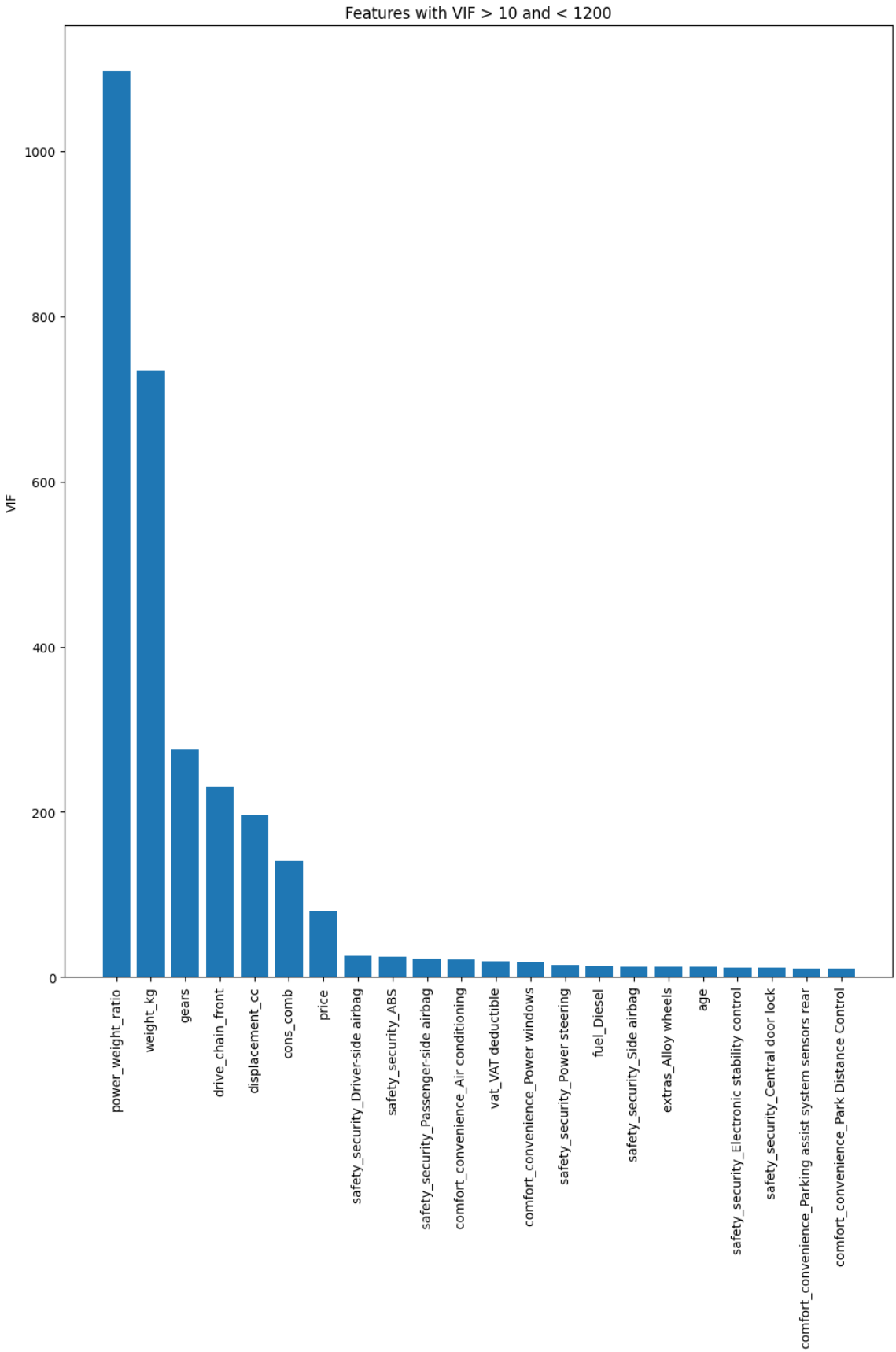


Few observations from the plot:

- Residuals are roughly centered around 0
- Shape is approximately bell-shaped,
- But there is a:
    - slight skewness where the left tail is longer
    - A few outliers (notably around -0.6)
    - Slight peaked-ness / non-perfect normality

Combined analysis:

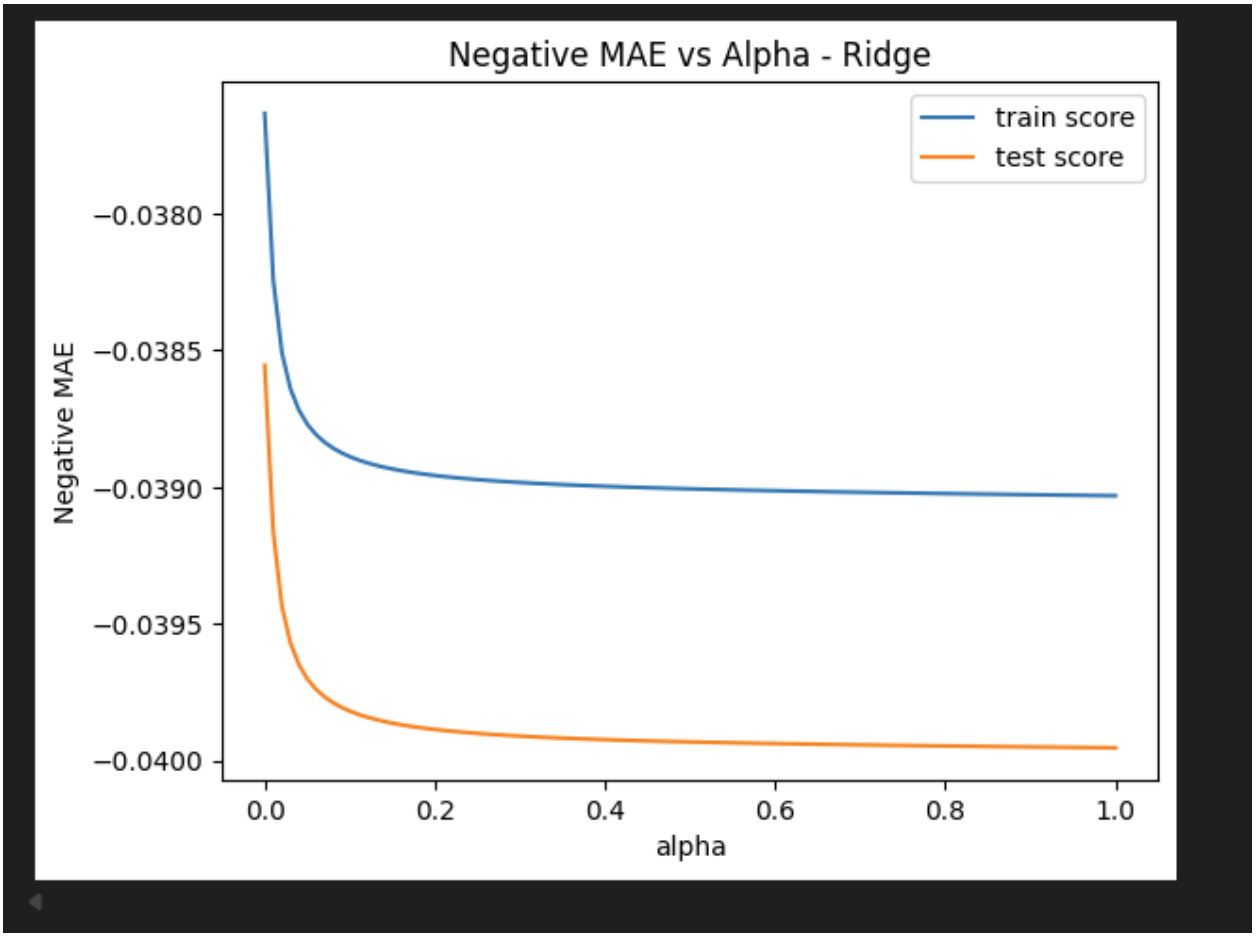
- Because the model overpredicts at extremes and underpredicts in the middle in the Test vs Train scatter plot
- We are getting systematic negative residuals in some regions
- This creates the longer left tail in the histogram


Features with VIF > 10 and < 1200
VIF
0
200
400
600
800
1000
power_weight_ratio
weight_kg
gears
drive_chain_front
displacement_cc
cons_comb
price
safety_security_Driver-side airbag
safety_security_ABS
safety_security_Passenger-side airbag
comfort_convenience_Air conditioning
vat_VAT deductible
comfort_convenience_Power windows
safety_security_Power steering
fuel_Diesel
safety_security_Side airbag
extras_Alloy wheels
age
safety_security_Electronic stability control
safety_security_Central door lock
comfort_convenience_Parking assist system sensors rear
comfort_convenience_Park Distance Control

```
Removing 'weight_kg' with VIF=265.73
Removing 'gears' with VIF=187.20
Removing 'displacement_cc' with VIF=169.15
Removing 'drive_chain_front' with VIF=130.79
Removing 'cons_comb' with VIF=102.09
Removing 'power_weight_ratio' with VIF=64.55
Removing 'price' with VIF=31.60
Removing 'safety_security_Driver-side airbag' with VIF=25.21
Removing 'safety_security_ABS' with VIF=23.49
Removing 'comfort_convenience_Air conditioning' with VIF=19.39
Removing 'comfort_convenience_Power windows' with VIF=17.02
Removing 'safety_security_Passenger-side airbag' with VIF=16.15
Removing 'vat_VAT deductible' with VIF=14.72
Removing 'safety_security_Power steering' with VIF=13.06
Removing 'comfort_convenience_Parking assist system sensors rear' with VIF=10.67
Removing 'safety_security_Side airbag' with VIF=10.52
                                       Predictor       VIF
0                                             km  6.004606
1                                            age  9.841323
2                     comfort_convenience_Armrest  4.069454
3   comfort_convenience_Automatic climate control  4.360720
4              comfort_convenience_Cruise control  5.791506
..                                            ...       ...
80                    gearing_type_Semi-automatic  1.455429
81                               drive_chain_rear  1.021624
82                               age_category_Mid       NaN
83                               age_category_Old       NaN
84                          age_category_Very Old       NaN

[85 rows x 2 columns]
```

3.2. Ridge Regression Implementation

3.2.1. Define a list of random alpha values



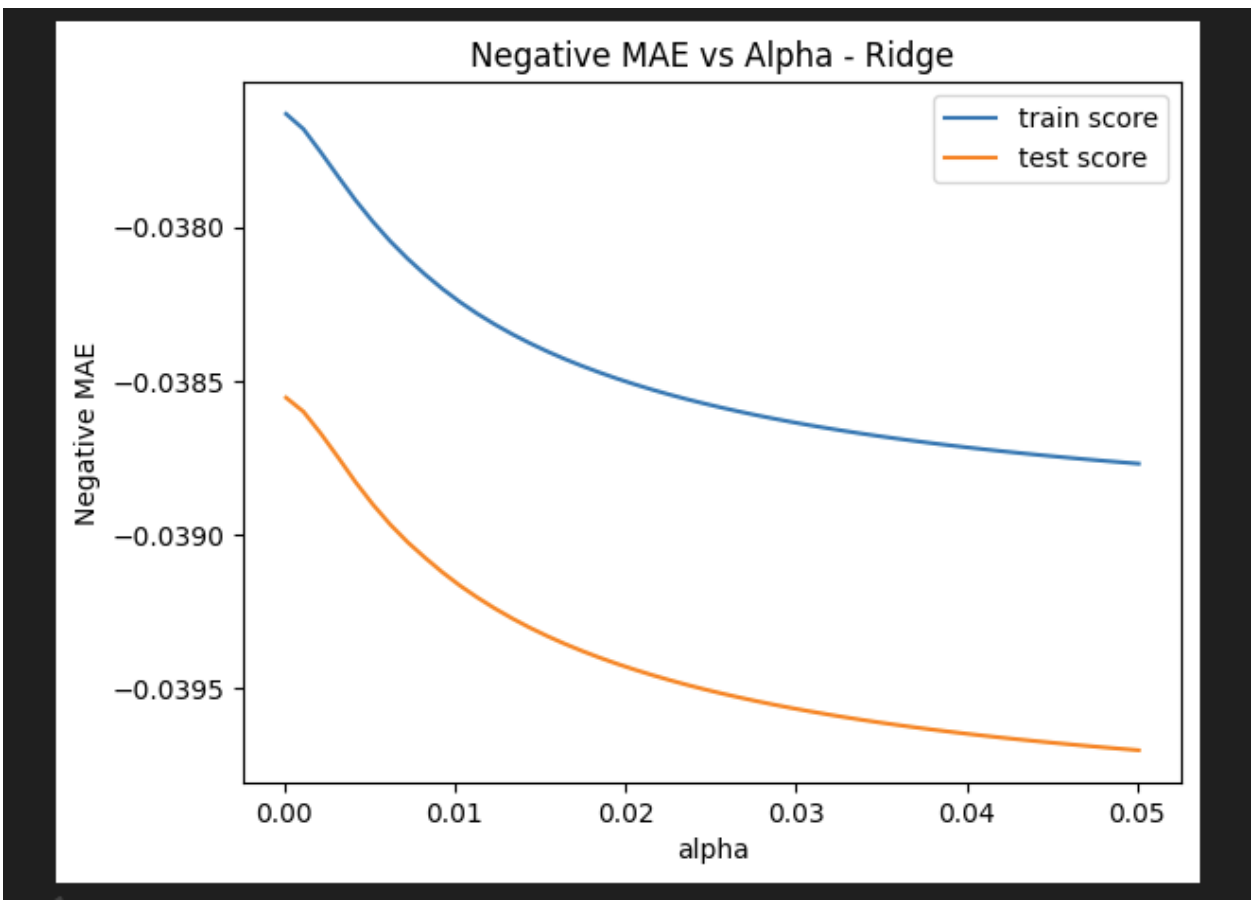
3.2.2. Apply Ridge Regularisation and find the best value of alpha from the list

```
Best alpha: 0.0001
Best negative MAE: -0.03855509146887389
```

We will get some best value of alpha above. This however is not the most accurate value but the best value from the given list. Now we have a rough estimate of the range that best alpha falls in. Let us do another iteration over the values in a smaller range.

	Feature	Coefficients - LR	Coefficients - Ridge
0	price	0.00	0.00
1	km	-0.00	-0.00
2	gears	-0.00	-0.00
3	age	-0.02	-0.02
4	hp_kw	-0.01	-0.01
...	...	...	...
97	drive_chain_front	0.02	0.02
98	drive_chain_rear	0.03	0.03
99	age_category_Mid	0.00	0.00
100	age_category_Old	0.00	0.00
101	age_category_Very Old	0.00	0.00

3.2.3. Fine tune by taking a closer range of alpha based on the previous result



```
Best alpha (fine search): 0.0001
Best negative MAE (fine search): -0.03855509146887389
```

	Feature	Ridge Coefficient
0	price	0.00
1	km	-0.00
2	gears	-0.00
3	age	-0.02
4	hp_kw	-0.01
...	...	...
97	drive_chain_front	0.02
98	drive_chain_rear	0.03
99	age_category_Mid	0.00
100	age_category_Old	0.00
101	age_category_Very Old	0.00

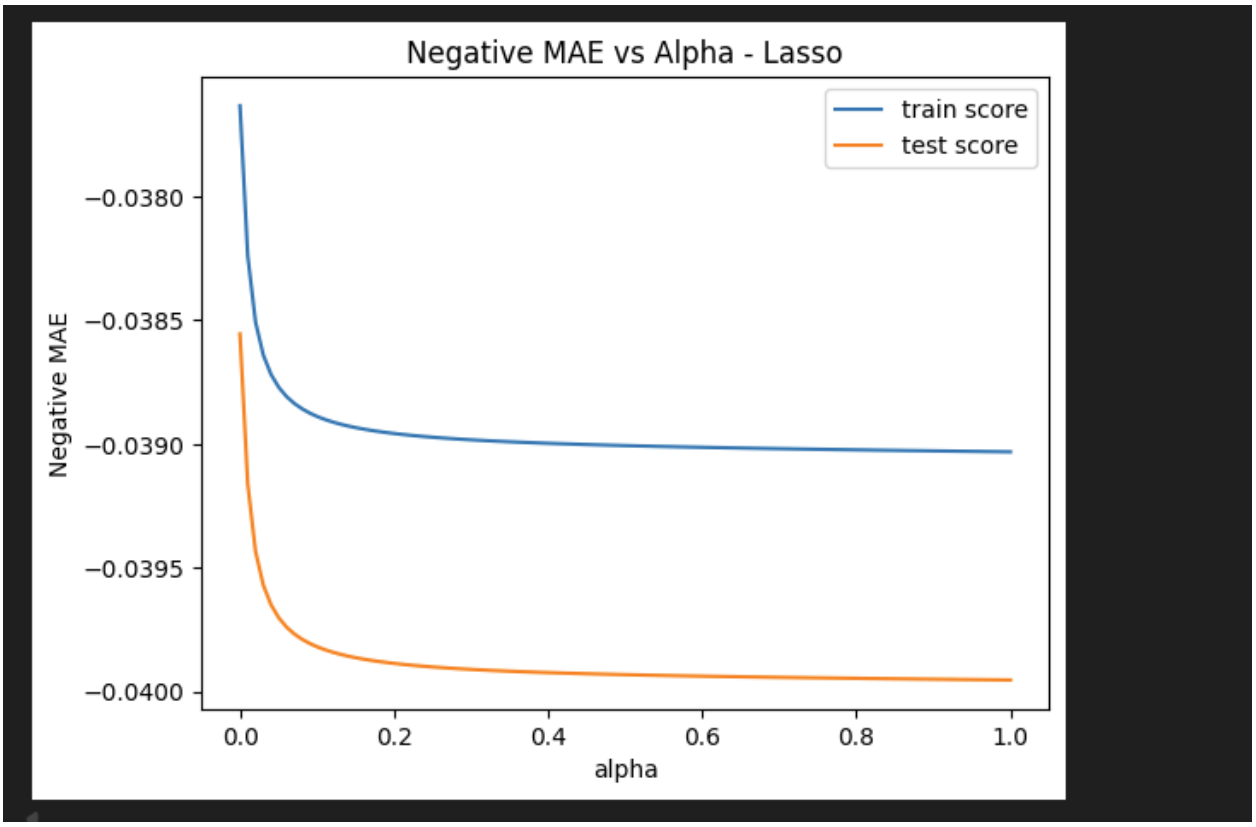
```
Ridge Test RMSE: 0.050
Ridge Test MAE: 0.038
Ridge Test R^2: 0.982
```

3.3. Lasso Regression Implementation

3.3.1. Define a list of random alpha values

```
Fitting 5 folds for each of 100 candidates, totalling 500 fits
94
```

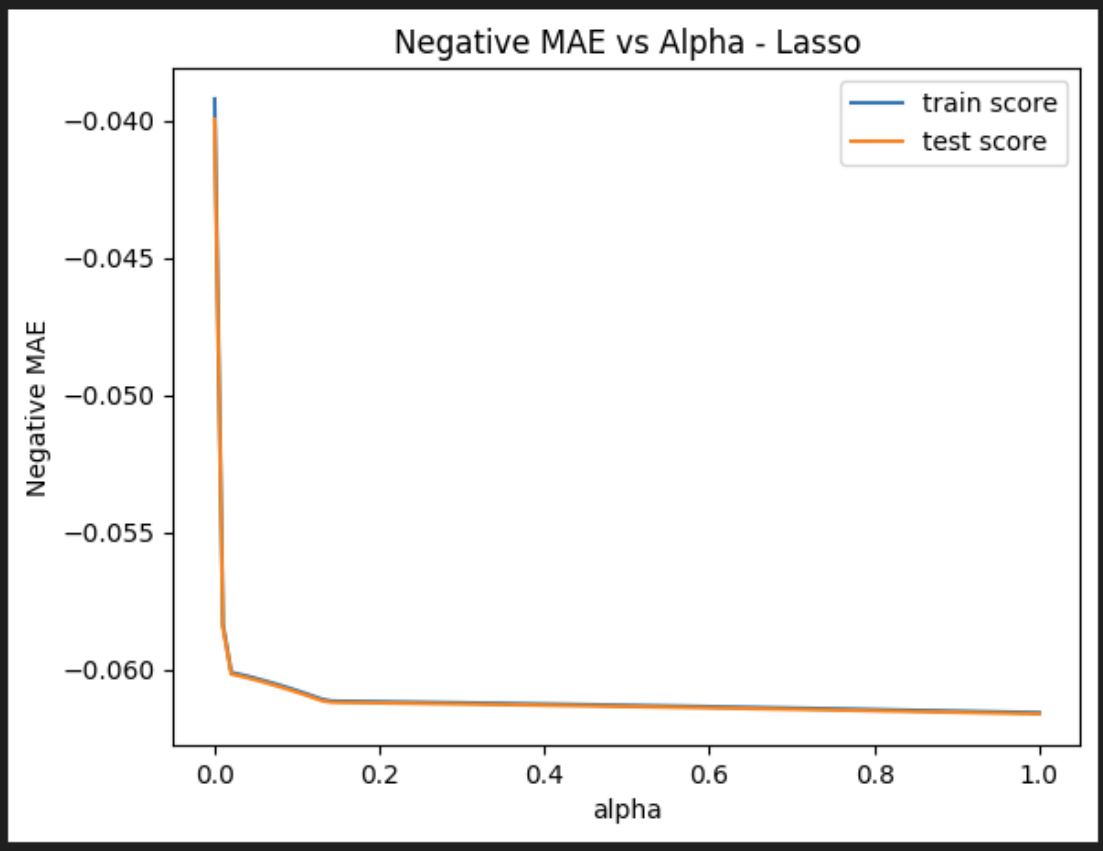
3.3.2. Apply Lasso Regularisation and find the best value of alpha from the list




```
Best alpha: 0.0001
Best negative MAE: -0.03995432237622539
```

3.3.3. Fine tune by taking a closer range of alpha based on the previous result

```
Fitting 5 folds for each of 100 candidates, totalling 500 fits
Best Lasso alpha: 0.0001
Best negative MAE (Lasso): -0.03995432237622539
```




```
Best Lasso alpha: 0.0001
Best negative MAE (Lasso): -0.03995432237622539
```

	Feature	Coefficients - Lasso
0	price	0.00
1	km	-0.00
2	gears	0.00
3	age	-0.02
4	hp_kw	0.00
...	...	...
97	drive_chain_front	0.01
98	drive_chain_rear	-0.00
99	age_category_Mid	0.00
100	age_category_Old	0.00
101	age_category_Very Old	0.00

3.4. Regularisation Comparison & Analysis

3.4.1. Compare the evaluation metrics for each model

	Model	RMSE	MAE	R^2
0	Linear Regression	0.0501	0.0380	0.9823
1	Ridge	0.0501	0.0380	0.9823
2	Lasso	0.0525	0.0399	0.9806

3.4.2. Compare the coefficients for the three models

Top 5 Linear Regression coefficients:

- power_weight_ratio 11.758838
- make_model_Opel Corsa 0.204346
- make_model_Renault Clio 0.169152
- make_model_Renault Espace 0.107370
- make_model_Audi A2 0.066087

Top 5 Ridge coefficients:

- power_weight_ratio 11.629635
- make_model_Opel Corsa 0.204402
- make_model_Renault Clio 0.169182
- make_model_Renault Espace 0.106772

- make_model_Audi A2 0.066353

Top 5 Lasso coefficients:

- make_model_Opel Corsa 0.196972
- make_model_Renault Clio 0.162966
- make_model_Opel Astra 0.058974
- gearing_type_Manual 0.046381
- body_type_Other 0.041583

Features eliminated by Lasso: 24
['power_weight_ratio', 'comfort_convenience_Automatic climate control', 'comfort_convenience_Hill Holder', 'comfort_convenience_Light sensor', 'comfort_convenience_Lumbar support', 'comfort_convenience_Parking assist system sensors front', 'comfort_convenience_Tinted windows', 'entertainment_media_Digital radio', 'extras_Catalytic Converter', 'safety_security_ABS', 'safety_security_Alarm system', 'safety_security_Electronic stability control', 'safety_security_Head airbag', 'safety_security_Isofix', 'safety_security_Side airbag', 'safety_security_Xenon headlights', 'make_model_Audi A2', 'vat_VAT deductible', 'fuel_Diesel', 'fuel_Other', 'drive_chain_rear', 'age_category_Mid', 'age_category_Old', 'age_category_Very Old']

4. Conclusions and key takeaways

4.1. Conclude with outcomes and insights gained

- Linear Regression already delivered very strong performance on the test set, and Ridge Regression produced almost identical results. This shows that regularization did not materially improve predictive accuracy for this dataset.
- The best Ridge alpha was very close to zero, indicating that only minimal shrinkage was required. This suggests the baseline linear model was already reasonably stable after preprocessing and feature engineering.
- Lasso gave slightly weaker predictive performance than Linear Regression and Ridge, but it improved interpretability by shrinking several coefficients to zero. This makes Lasso useful for feature selection, even though it was not the best-performing model in terms of accuracy.
- There is no major evidence of overfitting in the final model. The training and testing metrics are very similar, and the residual patterns for train and test data are also broadly consistent, which suggests the model generalizes well.

- - However, the residual diagnostics show that the linear model does not capture the relationship perfectly. The residual plots suggest some systematic pattern, with underprediction in the middle range and overprediction at the extremes, along with mild heteroscedasticity. This points to some remaining model bias and possible non-linear effects in the data.
- Overall, the available data is sufficient to build a strong predictive model for car prices, and a linear model works well as a solid baseline. However, based on residual behavior, a more flexible non-linear model may capture the remaining structure in the data better than regularized linear models.